

(12) **United States Patent**
Ponomarenko et al.

(10) **Patent No.: US 12,417,305 B1**
(45) **Date of Patent: Sep. 16, 2025**

(54) **DELEGATED AUTHORIZATION FOR CONSUMERS OF SHARED DATABASES**

(71) Applicant: **Amazon Technologies, Inc.**, Seattle, WA (US)

(72) Inventors: **Vladimir Ponomarenko**, Redwood City, CA (US); **Jingyi Qing**, Newcastle, WA (US); **Eric Ray Hotinger**, Redmond, WA (US); **William Michael McCreedy**, Berlin (DE); **Ippokratis Pandis**, Menlo Park, CA (US); **Pavel Sokolov**, Menlo Park, CA (US); **Mengyang Li**, Boston, MA (US); **Weihsang Song**, Bothell, WA (US); **Vivek Ramamoorthy**, Sammamish, WA (US); **Neeraja Rentachintala**, San Jose, CA (US); **Mohammad Foyzur Rahman**, Newark, CA (US)

(73) Assignee: **Amazon Technologies, Inc.**, Seattle, WA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 318 days.

(21) Appl. No.: **17/548,283**

(22) Filed: **Dec. 10, 2021**

(51) **Int. Cl.**
H04L 29/00 (2006.01)
G06F 16/23 (2019.01)
(Continued)

(52) **U.S. Cl.**
CPC **G06F 21/6218** (2013.01); **G06F 16/2358** (2019.01); **G06F 16/245** (2019.01); **G06F 16/252** (2019.01)

(58) **Field of Classification Search**
CPC G06F 21/6218; G06F 16/245; G06F 16/2358; G06F 16/252

(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

7,630,986 B1 * 12/2009 Herz G06Q 30/0603
707/999.009

9,350,801 B2 5/2016 Parakh
(Continued)

OTHER PUBLICATIONS

U.S. Appl. No. 17/548,414, filed Dec. 10, 2021, Naresh Chainani, et al.

(Continued)

Primary Examiner — Jeffrey C Pwu

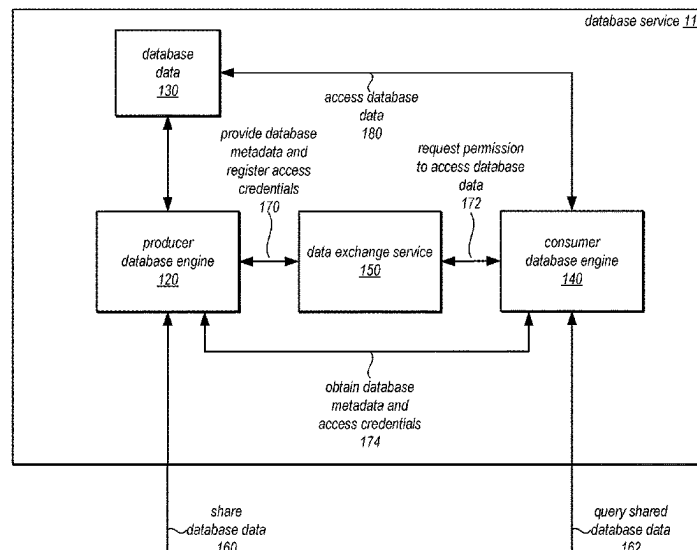
Assistant Examiner — Michael D Anderson

(74) *Attorney, Agent, or Firm* — S. Scott Foster; Kowert, Hood, Munyon, Rankin & Goetzel, P.C.

(57) **ABSTRACT**

A system for delegation of authorization management of a shared database of a database service is described. The database service includes a control plane and a producer database engine. The producer database engine receives a creation request to create a datashare for a database. The creation request delegates, to a data exchange service, authorization management to access the datashare. The control plane is configured to update permission data for the datashare to indicate that authorization to the datashare is managed by the data exchange service. The producer database engine is further configured to receive a request, from a consumer database engine, to obtain metadata used to perform a query to the database of the datashare, determine that the data exchange service authorized the consumer database engine to access the datashare based on the permission data for the datashare, and return the metadata to the consumer database engine.

20 Claims, 10 Drawing Sheets



Page 2

OTHER PUBLICATIONS

U.S. Appl. No. 17/548,406, filed Dec. 10, 2021, Ippokratis Pandis, et al.

* cited by examiner

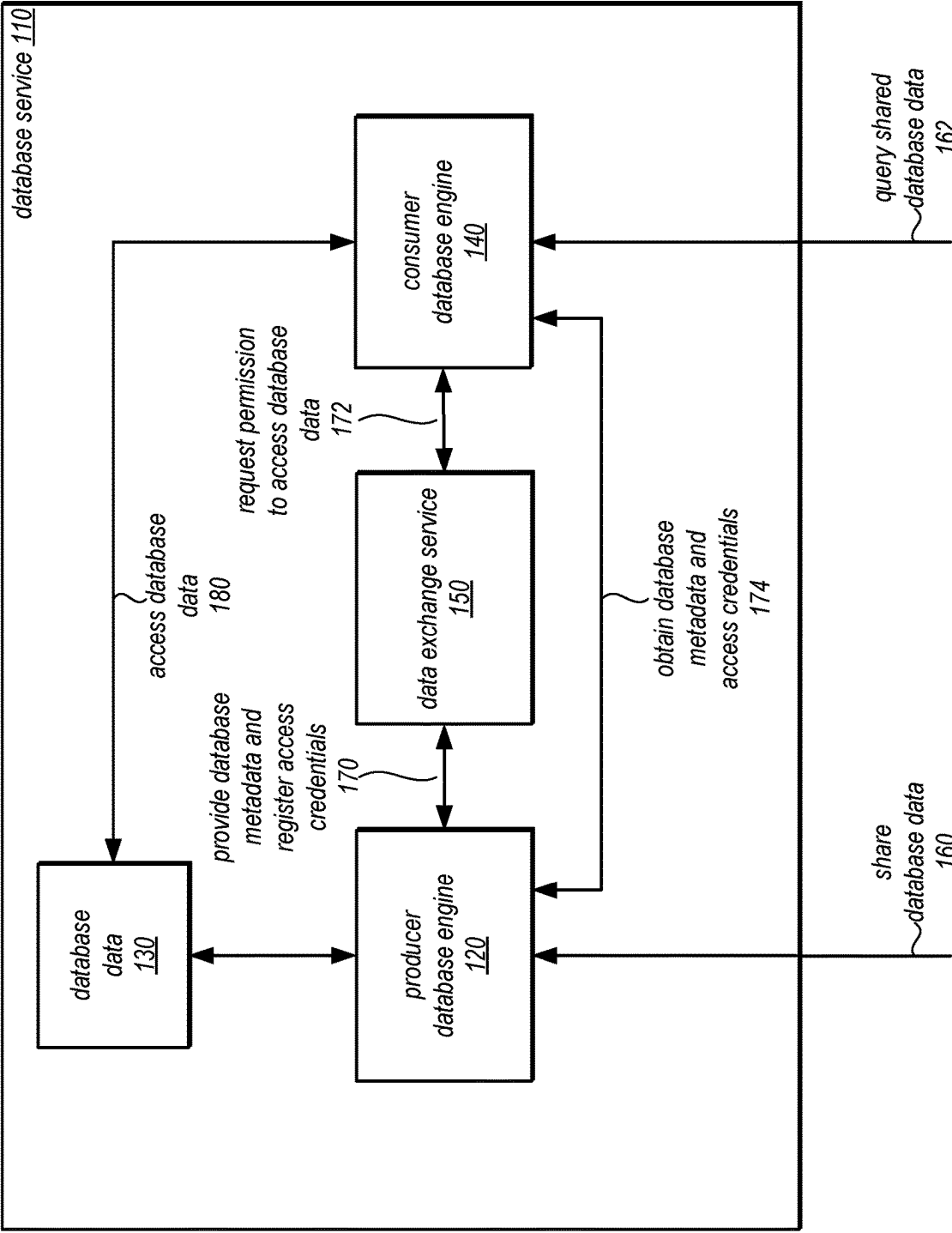


FIG. 1

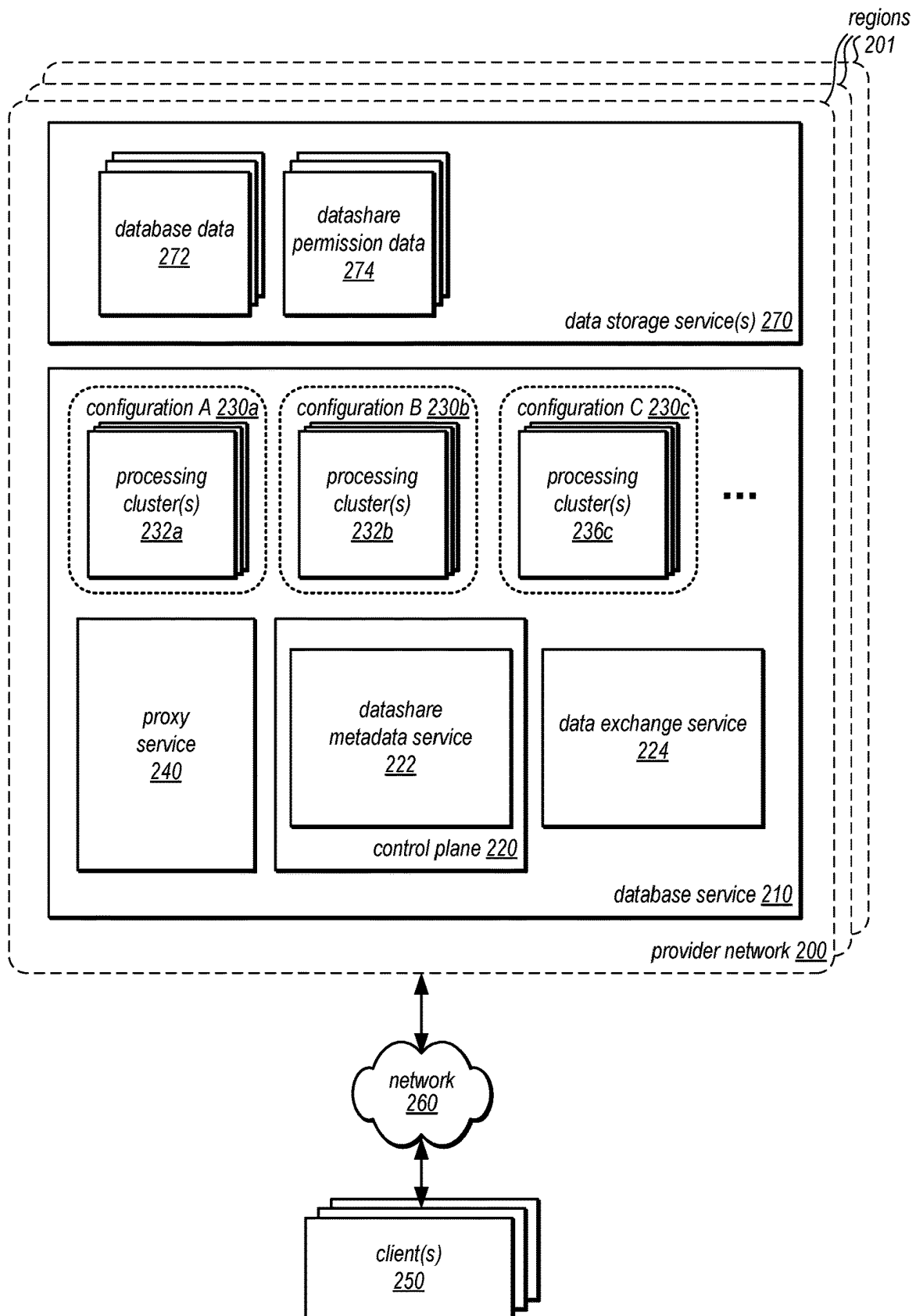


FIG. 2

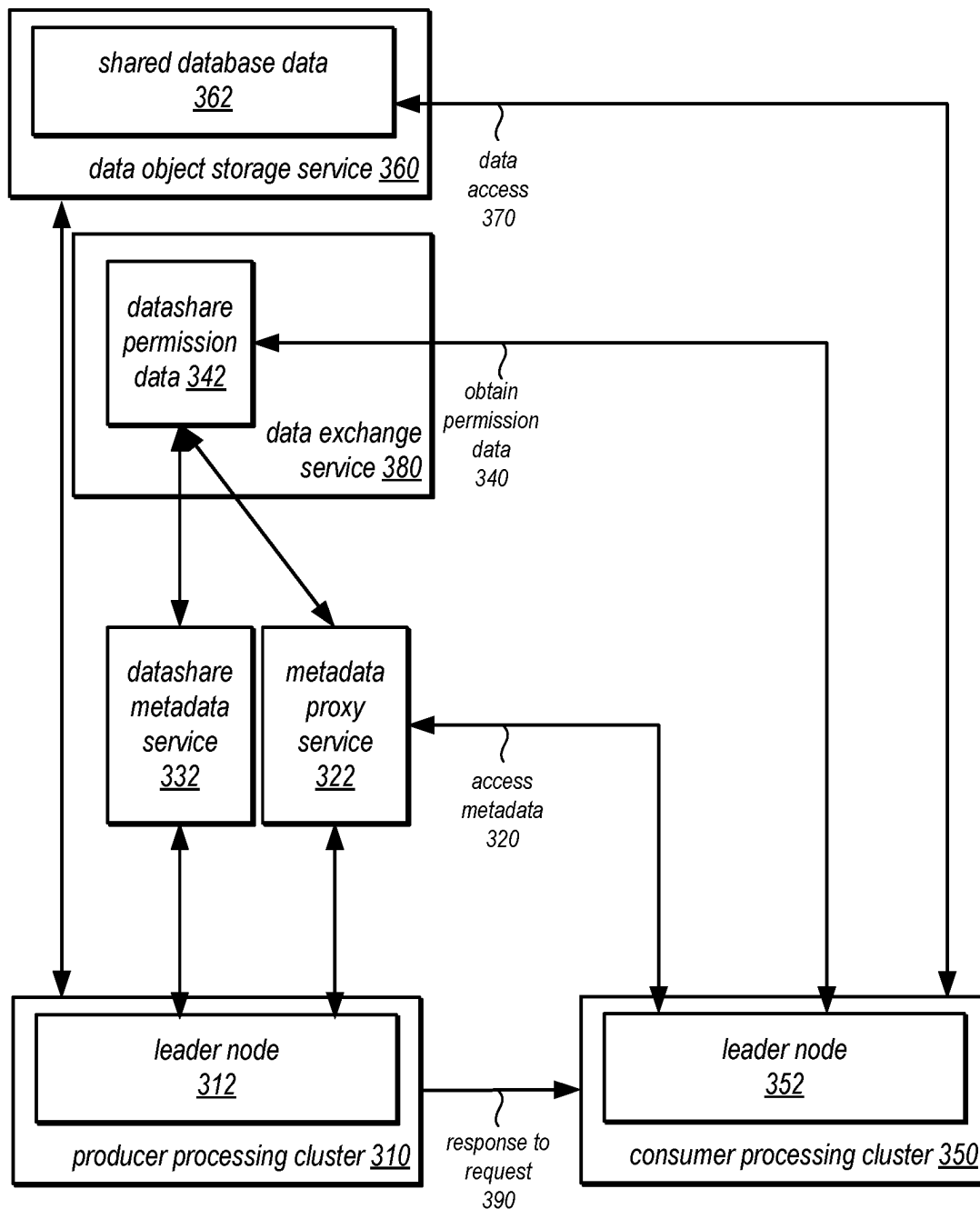


FIG. 3

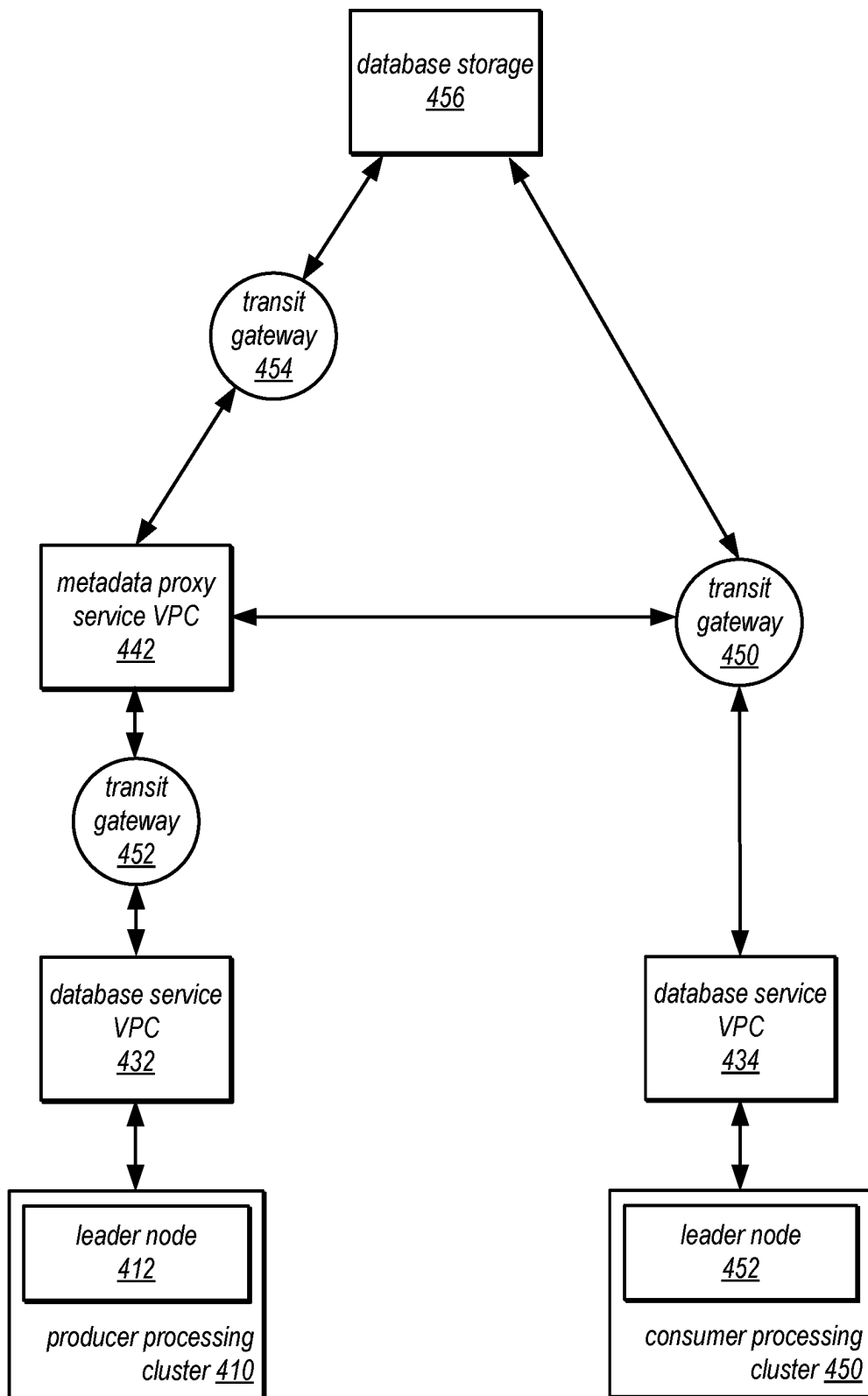


FIG. 4

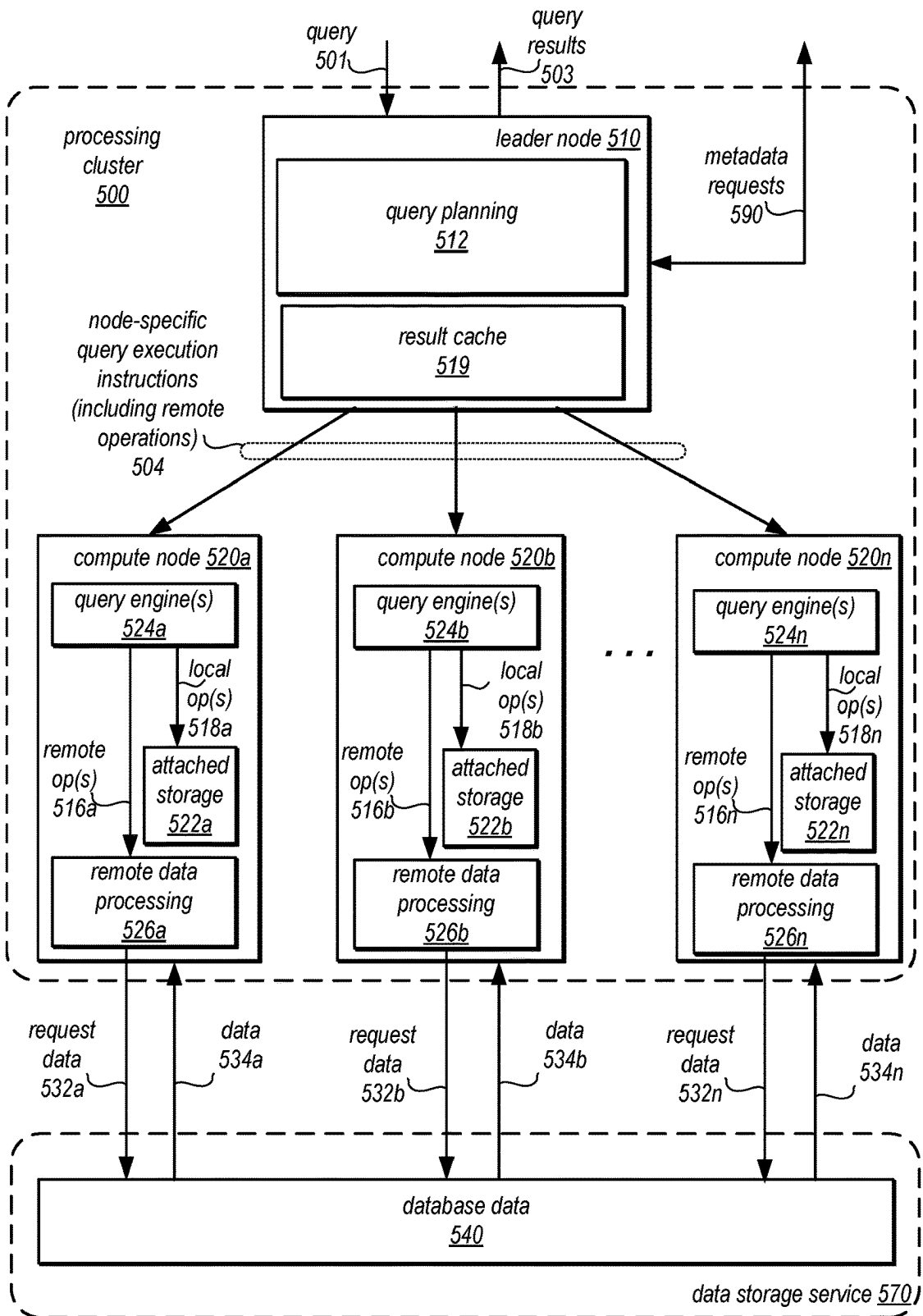


FIG. 5

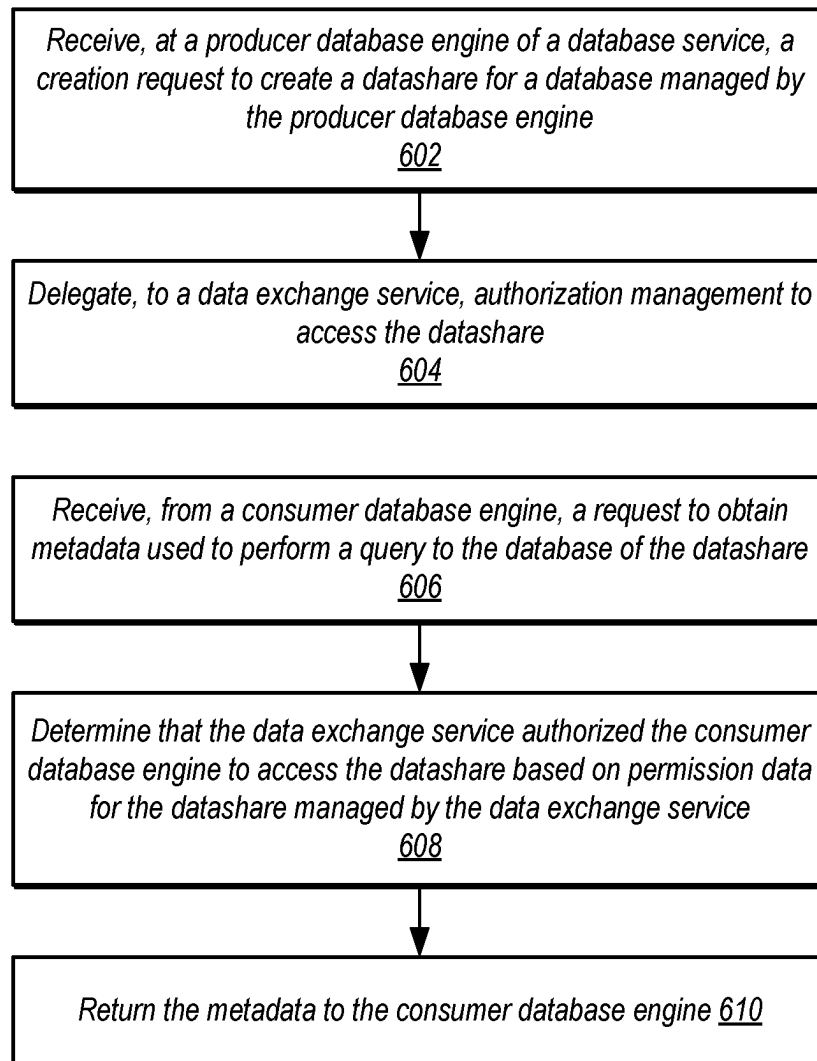
600
↘

FIG. 6

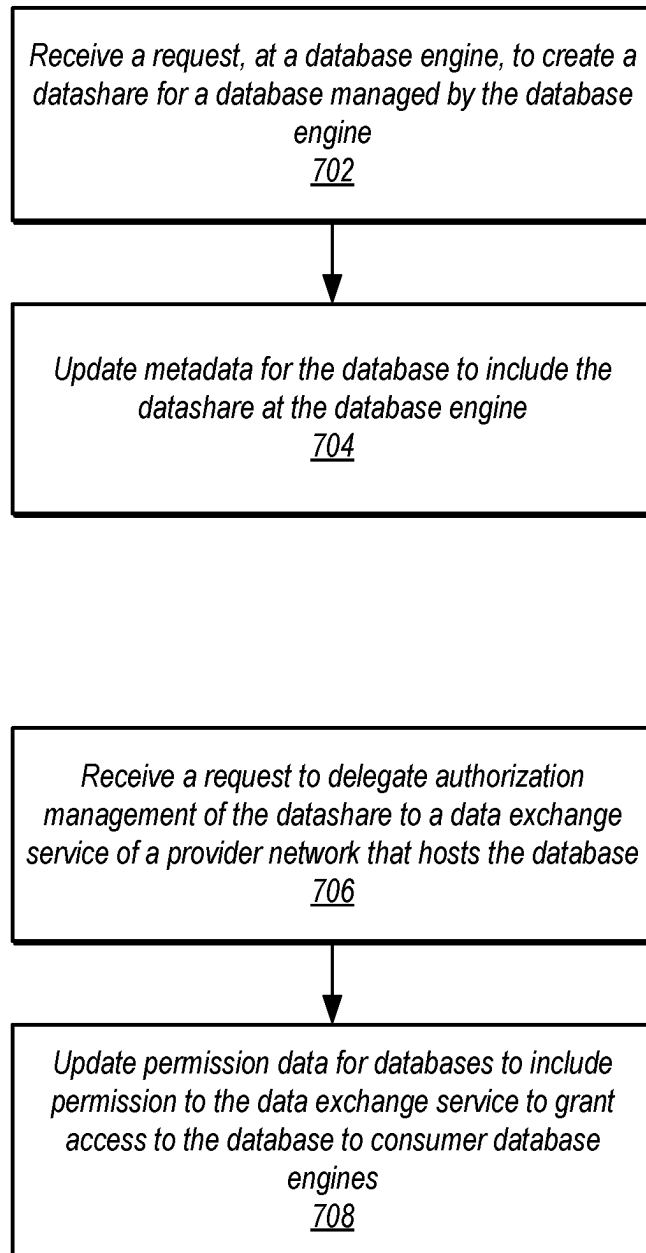
700
↘

FIG. 7

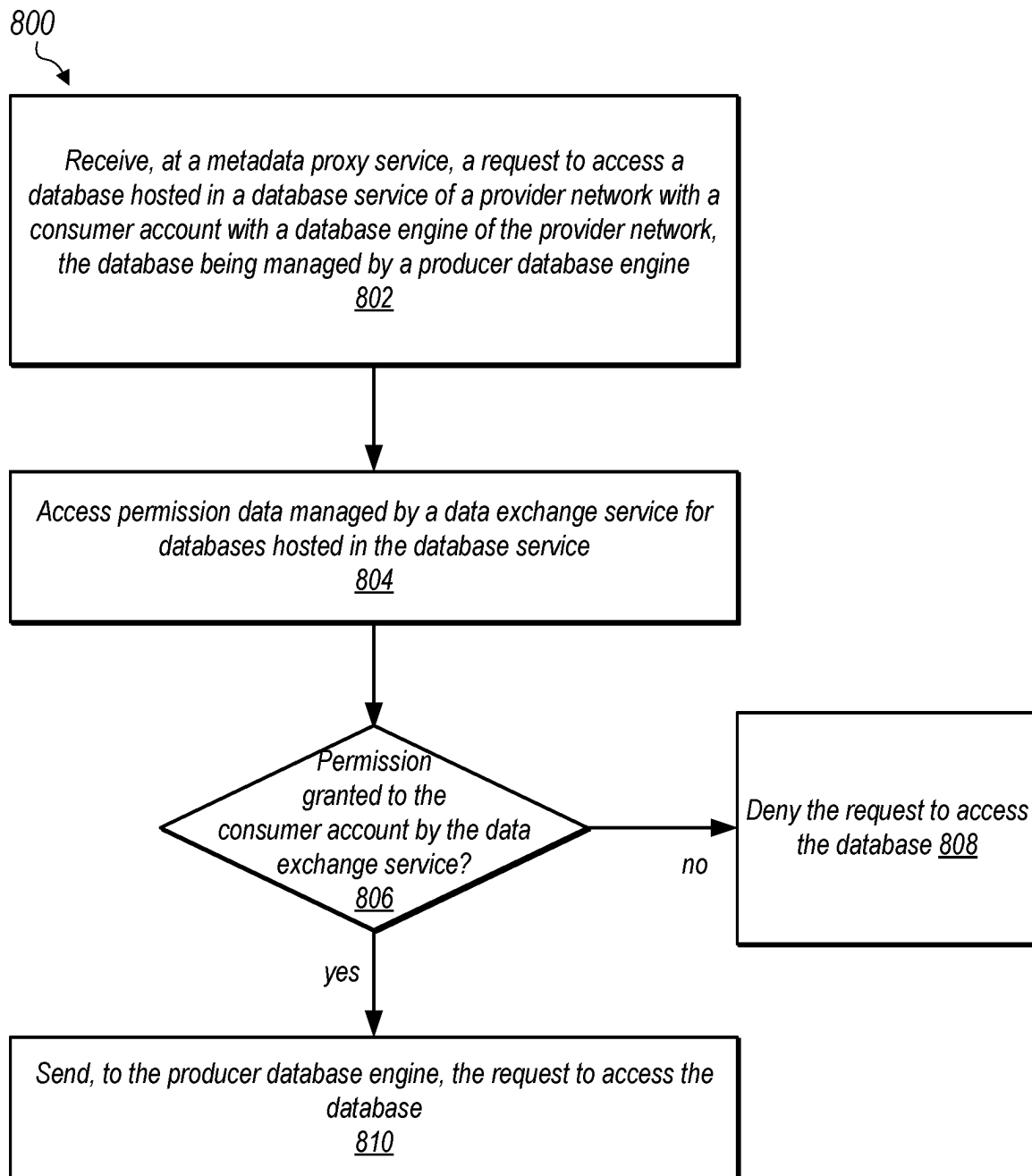


FIG. 8

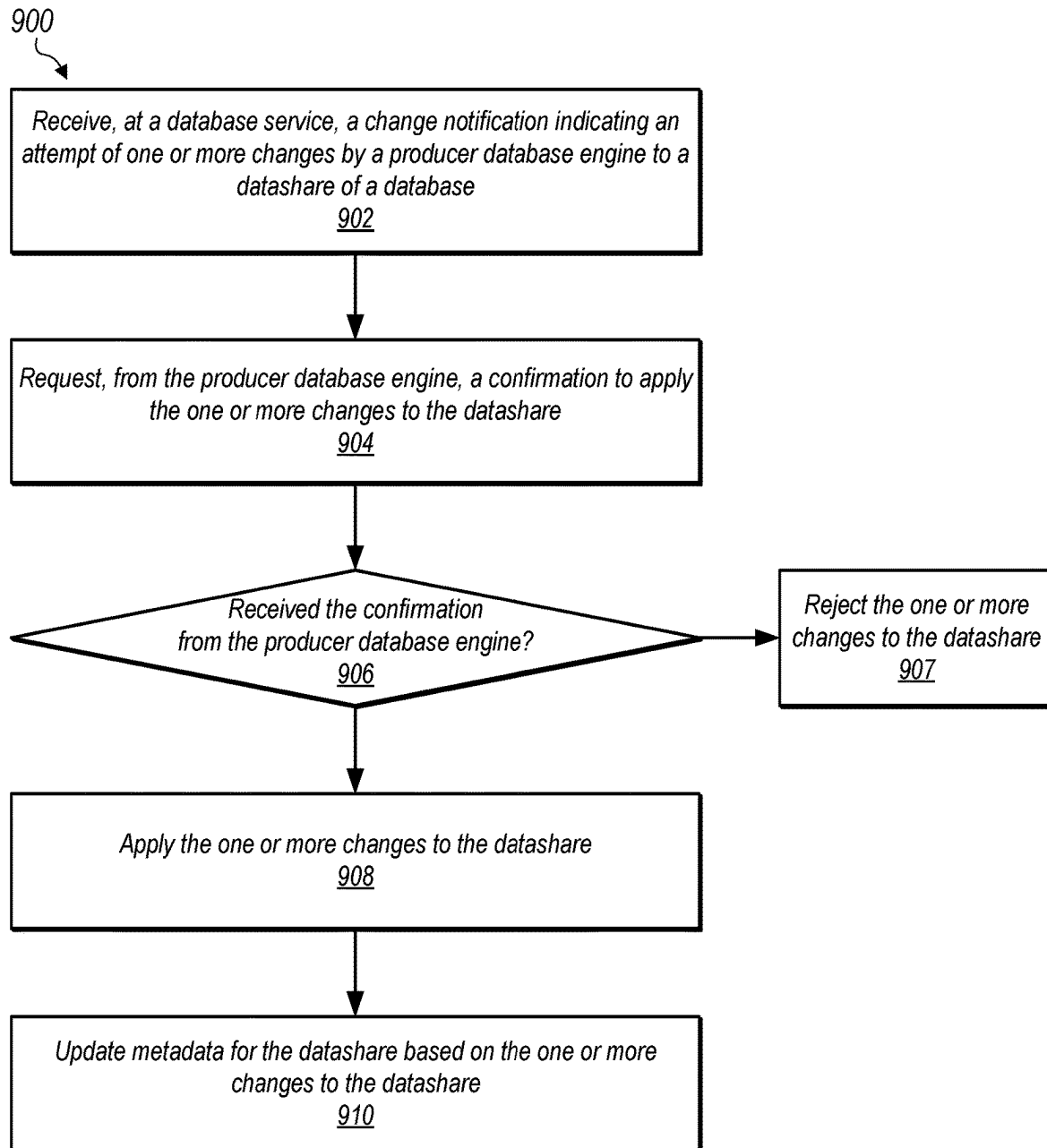


FIG. 9

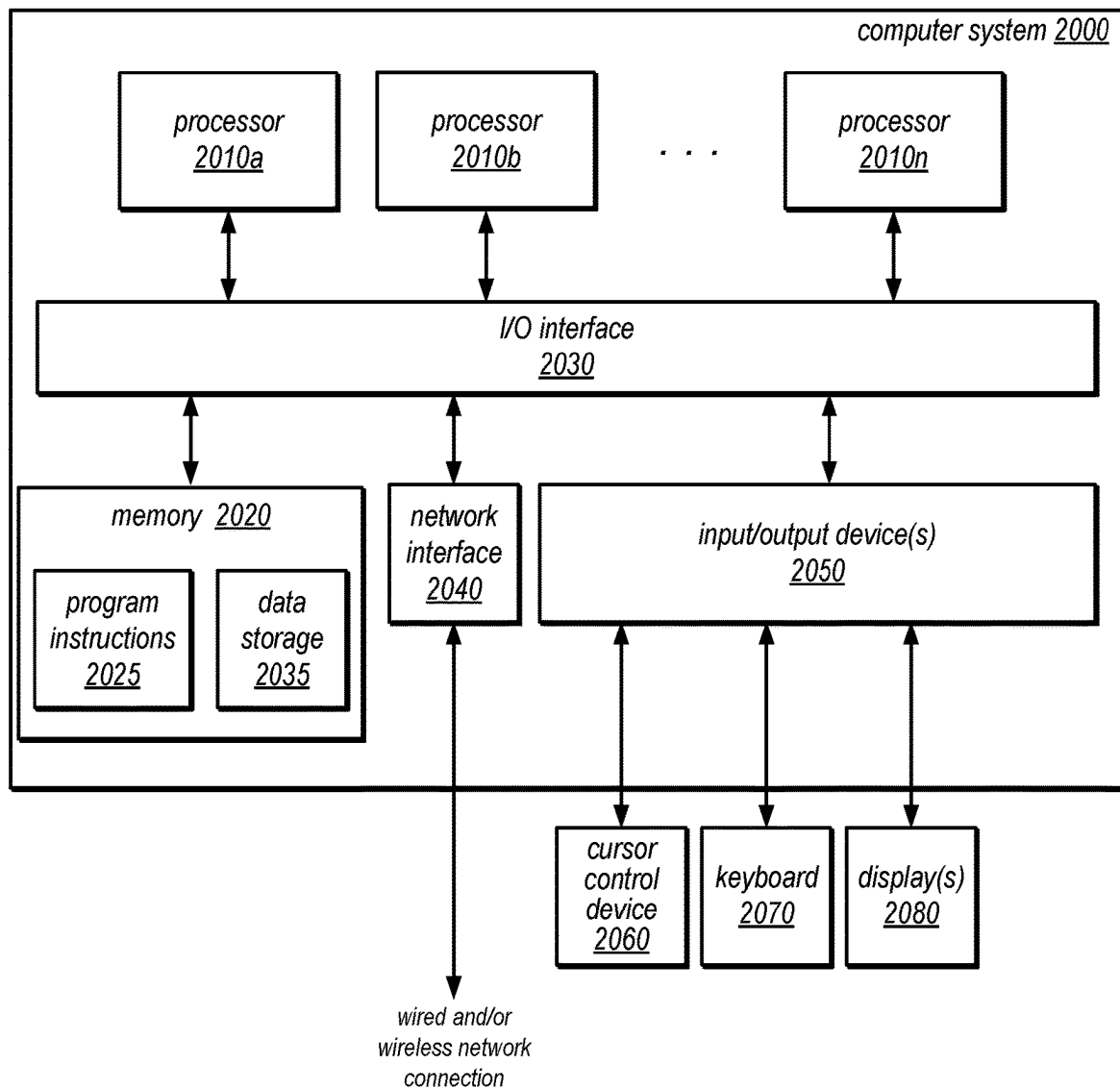


FIG. 10

1

DELEGATED AUTHORIZATION FOR CONSUMERS OF SHARED DATABASES

BACKGROUND

Clients of computing service providers may utilize computing resources to offer additional services to downline customers. A client may store data in a database that may be accessible to the client's customers. The client may establish an authentication system that allows the customers to access the data in the database. The authentication system may be implemented and deployed by the client. The client would have to maintain the authentication system to ensure that customers are properly added or removed as needed.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 illustrates a logical block diagram illustrating privately sharing database data across provider network regions, according to some embodiments.

FIG. 2 is a logical block diagram illustrating a provider network offering a database service that scales query processing resources for efficiently utilization performance for databases managed by the database service, according to some embodiments.

FIG. 3 is a logical block diagram illustrating interactions between control plane components to share database data, according to some embodiments.

FIG. 4 is a logical block diagram illustrating a network architecture for implementing communications between leader nodes to share database metadata, according to some embodiments.

FIG. 5 is a logical block diagram illustrating an example processing cluster of a database service, according to some embodiments.

FIG. 6 is a flowchart diagram illustrating a method for establishing a datashare for a database, according to some embodiments.

FIG. 7 illustrates a flowchart diagram for a method for delegating authorization management to a data exchange service, according to some embodiments.

FIG. 8 illustrates a flowchart diagram for a method for managing requests to access a shared database, according to some embodiments.

FIG. 9 illustrates a flowchart diagram depicting a method for confirming application of changes to a datashare, according to some embodiments.

FIG. 10 illustrates a block diagram illustrating an example computer system that may be used in some embodiments.

While embodiments are described herein by way of example for several embodiments and illustrative drawings, those skilled in the art will recognize that the embodiments are not limited to the embodiments or drawings described. It should be understood that the drawings and detailed description thereto are not intended to limit embodiments to the particular form disclosed, but on the contrary, the intention is to cover all modifications, equivalents and alternatives falling within the spirit and scope as defined by the appended claims. As used throughout this application, the word "may" is used in a permissive sense (i.e., meaning having the potential to), rather than the mandatory sense (i.e., meaning must). Similarly, the words "include", "including", and "includes" mean including, but not limited to.

This specification includes references to "one embodiment" or "an embodiment." The appearances of the phrases "in one embodiment" or "in an embodiment" do not necessarily refer to the same embodiment. Particular features,

2

structures, or characteristics may be combined in any suitable manner consistent with this disclosure.

DETAILED DESCRIPTION

5

Various systems and methods for managing shared data access are described herein. A data producer may establish a data share for data stored in a database. The data may be provided by the data producer to be stored in the database. A data share service may be established to manage access to the data stored in the database by data consumers. The data producer may provide access to data consumers via the data share service. The data producer may indicate that the data share service may delegate at least a portion of the data consumers may be managed by a data exchange service.

The data exchange service may be configured to manage the data consumers such that the data producer and the data share service are not responsible for controlling access by the data consumers. The data exchange service may take responsibility for providing access to the data consumers that may consolidate the data consumers as being a part of the data exchange service. For example, the data consumers accessing the data via the data exchange service may be exposed to the data share service as part of the data exchange service.

The data exchange service may be configured to maintain a list of the data consumers that is unmodifiable by the data share service or the data producer. By delegating management of the data consumers to the data exchange service, the data producer may be unable to add or remove permissions for at least a portion of the data consumers from accessing the data. The data exchange service may also add or remove data consumers without specific authorization from the data producer to add or remove data consumers.

The data producer may attempt to apply a destructive operation to at least a portion of the data from the database. The data share service may implement one or more operations to prevent the data producer from deleting the portion of the data based on one or more data integrity protocols. If the portion of the data is deleted, the data share service may be configured to send, to the data exchange service, an indication that the portion of data has been deleted. The data exchange service may restrict deletion of the portion of the data based on the data exchange service having delegated administrative control of the data.

FIG. 1 illustrates a logical block diagram illustrating privately sharing database data across provider network regions, according to some embodiments. Database service 110 may be a stand-alone database service, in various embodiments. For example, database service 110 may be implemented for private use (e.g., on private networks and resources for entity-specific utilization). In some embodiments, database service 110 may be implemented as part of multiple different services provided by a cloud service provider across multiple regions, such as provider network 200 discussed in detail below with regard to FIG. 2.

Database service 110 may manage databases on behalf of clients of database service 110, in various embodiments. For example, database service 110 may implement an interface that allows users to create a database to be hosted in database service 110. The interface may also allow users to specify whether the database is to be managed by the database service, automatically, in a "serverless" fashion (e.g., by allowing database service 110 to automatically determine and configure an appropriate number of computing resources to host and provide access to the database, such as querying the database). In some embodiments, the interface

may support management parameters or other information to guide the management of the database, such as a parameter indicating that query performance should be prioritized over resource efficiency (e.g., lower cost), or parameter to indicate that resource efficiency should be prioritized over query performance. In some embodiments, database service 110 may also allow for hosted databases to be manually managed by a user (e.g., via interface requests to configure a specified number of computing resources to host and provide access to (e.g., query) the database), increase or decrease the number of computing resources, and so on.

Database service 110 may be implemented across different regions in order to provide access to database service 110 in different locations. Regions, as discussed in detail below with regard to FIG. 2, may be separate geographical areas in which the provides data centers. Client applications can connect to regions via a publicly accessible network (e.g., the Internet, a cellular communication network). Regions may be connected to a global network which includes private networking infrastructure (e.g., fiber connections controlled by the database service) connecting each region to at least one other region. This compartmentalization and geographic distribution of computing hardware enables the database service 210 to provide low-latency resource access to customers on a global scale with a high degree of fault tolerance and stability.

A database may be created and hosted in by the database service 110. For example, a database may be created and hosted as database data 130. Database data 130 may be stored for the database in a data storage system (e.g., remote or attached to producer database engine). Database service 110 may implement a database engine 120 (e.g., one or more computing resources, such as a processing cluster discussed in detail below with regard to FIG. 5), which may manage and provide access to the database data 130. As discussed above, it may be desirable to share a database for use by other database engines. Thus, the database engine 120 may receive a request to share database data, as indicated 160. This makes database engine 120 have the role of “producer” for database data (e.g., as data may be added or removed to database data 130 via producer database engine 120). Sharing the database may create a “datashare” object (e.g., a logical object, such as an identifier for the datashare allowing the datashare to be referenced in various requests to manage or access the datashare).

Database data 130 may be stored and organized into one or more schemas (e.g., for one or more database tables). These schemas may indicate how to interpret database data 130 at a database engine (e.g., at producer database engine 120 or consumer database engine 140), such as by indicating what type of data is stored in a column, feature, or other attribute of database data. Other metadata may be used to access database data. For example, various statistics that describe the contents of database data (e.g., histograms of data values, minimum and maximum values, etc.) may also be stored as part of metadata. In some embodiments, the metadata may be organized in various data objects, such as a superblock, which may map portions of metadata to one (or more) data blocks in database data 130.

Database service 110 may include data exchange service 150 to manage consumer access to the database data 130. Producer database engine 120 may be configured to delegate authorization management of consumer access to the database data 130. Data exchange service 150 may grant or revoke access to the database data 130 by the consumer database engine 140. In some embodiments, consumer database engines may request access to the database data 130 by

sending requests to the data exchange service 150. For example, when the database exchange service 150 has been delegated authorization management, the consumer database engines may request authorization from the database exchange service 150 instead of requesting authorization from the producer database engine 120. In some implementations, the data exchange service 150 may be implemented as part of a datashare authentication manager configured to delegate permissions to data consumers.

Data exchange service 150 may receive requests from consumer database engine 140 for permission to access the database data 130, as indicated at 172. Data exchange service 150 may provide the requested permission based on having been delegated authorization management by producer database engine 120. In some embodiments, data exchange service 150 may be configured to provide access credentials to consumer database engine 140 for configuring the consumer database engine 140 to access the database data 130. For example, data exchange service 150 may be configured to generate the access credentials for accessing the database data 130. As another example, producer database engine 120 may be configured to provide access credentials to the data exchange service 150 to be distributed by data exchange service 150.

Once database data is shared, the metadata to access the database may be obtained. For example, data exchange service 150 may be authorized by producer database engine 120 to obtain the metadata to access the database. Data exchange service 150 may provide the metadata to consumer database engine 140. In some embodiments, consumer database engine 140 may be a database engine that has been authorized to access the shared database data. In some embodiments, database service 110 (or provider network 200) may implement accounts as an access control mechanism so that producer database engine 120 may be associated with one account and consumer database engine 140 may be associated with another account. Data exchange service 150 may be configured to associate the accounts of producer database engine 120 and consumer database engine 140 to determine that consumer database engine 140 is authorized to access the shared database data. In some embodiments, producer database engine 120 may acknowledge consumer database engine 140 as a constituent of data exchange service 150 as opposed to a separate, standalone account for the datashare.

In some embodiments, when data exchange service 150 is delegated authorization management by producer database engine 120, data exchange service 150 may be notified when producer database engine 120 attempts various operations with the database data 130. Producer database engine 120 may receive an instruction to perform operations that may result in at least a portion of database data 130 being made unavailable. For example, the operations may include destructive operations such as deleting at least a portion of database data 130, altering or disabling the datashare, modifying or deleting a schema for the database data 130, dropping a table from database data 130, dropping a view, dropping one or more functions for database data 130, altering a table structure for the datashare, altering a materialized view name, or any other operation that may result in removal or destruction of database data 130.

Producer database engine 120 may provide database metadata and register access credentials with data exchange service 150, as indicated at 170. The database metadata may be provided by the producer database engine 120 to data exchange service 150. The database metadata may indicate that database data 130 may be shared with one or more data

consumers. In some embodiments, data exchange service 150 may provide information indicating the consumers permitted to access the database data 130. Producer database engine 120 may register access credentials for the consumers based on information provided by data exchange service 150. Data exchange service 150 may associate or disassociate accounts with the datashare by sending additional information indicating consumers that are to be added or removed from the datashare.

The database metadata may provide an indication to data exchange service 150 as to what data is being shared with consumers from the database data 130. In some embodiments, the database metadata may be provided to consumer database engine 140 in response to a request for updated metadata for the database data 130.

Data exchange service 150 may provide indications of one or more consumers as part of associating accounts of the consumers with producer database engine 120. Information indicating the associated accounts may be stored as part of database data 130, in some embodiments. In other embodiments, the producer database engine 120 may detect that a user authorized by data exchange service 150 based on sending an authentication request to data exchange service 150.

Producer database engine 120 may include data integrity protocols configured to prevent database data 130 subject to the datashare from being deleted, removed or destructively modified. Producer database engine 120 may initially attempt to apply a change to the datashare responsive to a command from a client or producer administrator. In some embodiments, data exchange service 150 may be notified that the change to the datashare was attempted by producer database engine 120. Data exchange service 150 may request a confirmation from producer database engine 120 to apply the change to the datashare. In some embodiments, the request may include an error message with a one-time-use value to temporarily unlock administrative access to the datashare such that producer database engine 120 may retry the attempted change. In response to the request for confirmation, producer database engine 120 may send a confirmation to data exchange service 150. In some embodiments, the database service 110 may be configured to request the confirmation from producer database engine 120 as described herein. Producer database engine 120 may send a confirmation notification to a source of the one or commands, where the confirmation notification includes an indication of the changes to be applied, an indication of target data of the one or more changes to the datastore.

Consumer database engine 140 may receive a query to shared database data 162. Consumer database engine 140 may utilize network communications with data exchange service 150 to obtain database metadata and access credentials for database data, as indicated at 172. For example, as discussed in detail below with regard to FIG. 4, various private networking techniques, such as techniques that utilize logically isolated network communications (or physically isolated network communications) in order to avoid exposing metadata. In some embodiments, consumer database engine 140 and producer database engine 120 may not communicate directly, but instead through a proxy, as discussed in detail below with regard to FIG. 3. In this way, database engines can be isolated from potentially malicious actions or other failure events.

Consumer database engine 140 may request the database metadata and access credentials from producer database engine 120. Producer database engine 120 may be configured to determine whether the consumer database engine

140 is permitted to access the database data 130 based on determining whether consumer database engine 140 is associated with an account that is registered with producer database engine 120 in accordance with data exchange service 150.

To perform query 162, consumer database engine 140 may utilize the metadata to generate a query plan to perform the query, including various instructions, operations, or other steps to perform. Consumer database engine 140 may then request permission to access the shared database data 130 in order to perform the query, as indicated at 172.

Consumer database engine 140 may obtain the database metadata and access information from producer database engine 120, as illustrated in 174. Consumer database engine 140 may generate access requests in accordance with the database metadata and the access information to access the database data 130, as indicated at 180. In some embodiments, consumer database engine 140 may be configured to identify that consumer database engine 140 is attempting to access the database data 130 in accordance with data exchange service 150. In other embodiments, producer database engine 120 may be configured to determine whether the consumer database engine 140 is permitted to access the database data 130. For example, producer database engine 140 may be configured to determine whether the datashare is managed by data exchange service 150 and whether the consumer has a registered account with data exchange service 150. In some embodiments, consumer database engine 140 may utilize the same network communication techniques as used for metadata to obtain access to shared database data 130. Although not illustrated, a result to query 162 may be returned.

Please note that the previous description of a database service is a logical description and thus is not to be construed as limiting as to the implementation of database engines, a database service, database data, regions, and performance of queries, or portions thereof.

This specification continues with a general description of a provider network that implements multiple different services, including a database service and storage service, which may implement scaling query processing resources for efficient utilization and performance. Then various examples of the database service and storage service, including different components/modules, or arrangements of components/module that may be employed as part of implementing the services are discussed. A number of different methods and techniques to implement scaling query processing resources for efficient utilization and performance are then discussed, some of which are illustrated in accompanying flowcharts. Finally, a description of an example computing system upon which the various components, modules, systems, devices, and/or nodes may be implemented is provided. Various examples are provided throughout the specification.

FIG. 2 is a logical block diagram illustrating a provider network offering a database service that scales query processing resources for efficiently utilization performance for databases managed by the database service, according to some embodiments. Provider network 200 may be a private or closed system or may be set up by an entity such as a company or a public sector organization to provide one or more services (such as various types of cloud-based storage) accessible via the Internet and/or other networks to clients 250.

Provider network 200 may be implemented in a single location or may include numerous data centers hosting various resource pools, such as collections of physical

and/or virtualized computer servers, storage devices, networking equipment and the like (e.g., computing system 2000 described below with regard to FIG. 12), needed to implement and distribute the infrastructure and storage services offered by the provider network 200. The provider network 200 can be formed as a number of regions (e.g., regions 201), where a region is a separate geographical area in which the cloud provider clusters data centers. Each region can include two or more availability zones connected to one another via a private high speed network, for example a fiber communication connection. An availability zone (also known as an availability domain, or simply a “zone”) refers to an isolated failure domain including one or more data center facilities with separate power, separate networking, and separate cooling from those in another availability zone. Preferably, availability zones within a region are positioned far enough away from one other that the same natural disaster should not take more than one availability zone offline at the same time. Customers can connect to availability zones of the provider network 200 via a publicly accessible network (e.g., the Internet, a cellular communication network).

Regions are connected to a global network which includes private networking infrastructure (e.g., fiber connections controlled by the cloud provider) connecting each region to at least one other region. The provider network 200 may deliver content from points of presence outside of, but networked with, these regions by way of edge locations and regional edge cache servers. An edge location can be an extension of the cloud provider network outside of the traditional region/AZ context. For example an edge location can be a data center positioned to provide capacity to a set of customers within a certain latency requirement, a set of servers provided to a customer’s premises, or a set of servers provided within (or forming part of) a cellular communications network, each of which can be controlled at least in part by the control plane of a nearby AZ or region. This compartmentalization and geographic distribution of computing hardware enables the provider network 200 to provide low-latency resource access to customers on a global scale with a high degree of fault tolerance and stability.

The traffic and operations of the provider network may broadly be subdivided into two categories in various embodiments: control plane operations carried over a logical control plane and data plane operations carried over a logical data plane. While the data plane represents the movement of user data through the distributed computing system, the control plane represents the movement of control signals through the distributed computing system. The control plane generally includes one or more control plane components distributed across and implemented by one or more control servers. Control plane traffic generally includes administrative operations, such as system configuration and management (e.g., resource placement, hardware capacity management, diagnostic monitoring, system state information). The data plane includes customer resources that are implemented on the cloud provider network (e.g., computing instances, containers, block storage volumes, databases, file storage). Data plane traffic generally includes non-administrative operations such as transferring customer data to and from the customer resources. Certain control plane components (e.g., tier one control plane components such as the control plane for a virtualized computing service) are typically implemented on a separate set of servers from the data plane servers, while other control plane components (e.g., tier two control plane components such as analytics services) may

share the virtualized servers with the data plane, and control plane traffic and data plane traffic may be sent over separate/distinct networks.

In some embodiments, provider network 200 may implement various computing resources or services, such as database service(s) 210, (e.g., relational database services, non-relational database services, a map reduce service, a data warehouse service, and/or other large scale data processing services or various other types database services), data storage service 270 (e.g., object storage services or block-based storage services that may implement a centralized data store for various types of data), and/or any other type of network based services (which may include a virtual compute service and various other types of storage, processing, analysis, communication, event handling, visualization, and security services not illustrated).

In various embodiments, the components illustrated in FIG. 2 may be implemented directly within computer hardware, as instructions directly or indirectly executable by computer hardware (e.g., a microprocessor or computer system), or using a combination of these techniques. For example, the components of FIG. 2 may be implemented by a system that includes a number of computing nodes (or simply, nodes), each of which may be similar to the computer system embodiment illustrated in FIG. 12 and described below. In various embodiments, the functionality of a given system or service component (e.g., a component of database service 210 or data storage service 270) may be implemented by a particular node or may be distributed across several nodes. In some embodiments, a given node may implement the functionality of more than one service system component (e.g., more than one data store component).

Database services 210 may be (or included in) various types of data processing services that perform general or specialized data processing functions (e.g., anomaly detection, machine learning, data mining, big data querying, or any other type of data processing operation). For example, in at least some embodiments, database services 210 may include a map reduce service that creates clusters of processing nodes that implement map reduce functionality over data stored in the map reduce cluster as well as data stored in data storage service 270. In another example, database service 210 may include various types of database services (both relational and non-relational) for storing, querying, and updating data. Such services may be enterprise-class database systems that are highly scalable and extensible. Queries may be directed to a database in database service 210 that is distributed across multiple physical resources, and the resource configurations, such as processing clusters 232, used to process the queries may be scaled up or down on an as needed basis.

Database service 210 may work effectively with database schemas of various types and/or organizations, in different embodiments. In some embodiments, clients/subscribers may submit queries in a number of ways, e.g., interactively via an SQL interface to the database system. In other embodiments, external applications and programs may submit queries using Open Database Connectivity (ODBC) and/or Java Database Connectivity (JDBC) driver interfaces to the database system. For instance, database service 210 may implement, in some embodiments, a data warehouse service, that utilizes another data processing service, to execute portions of queries or other access requests with respect to data that is stored in a remote data store, such as data storage service(s) 270 (or a data store external to

provider network 200) to implement distributed data processing for distributed data sets.

In at least some embodiments, database service 210 may be a data warehouse service. Thus in the description that follows, database service 210 may be discussed according to the various features or components that may be implemented as part of a data warehouse service, including control plane 220, proxy service 240, and processing clusters 232. Note that such features or components may also be implemented in a similar fashion for other types of database services and thus the following examples may be applicable to other types of database service 210. Database service 210 may implement one (or more) processing clusters that are attached to a database (e.g., a data warehouse). In some embodiments, these processing clusters may be designated as a primary and secondary (or concurrent, additional, or burst processing clusters) that perform queries to an attached database warehouse.

In embodiments where database service 210 is a data warehouse service, the data warehouse service may offer clients a variety of different data management services, according to their various needs. In some cases, clients may wish to store and maintain large amounts of data, such as sales records, marketing, management reporting, business process management, budget forecasting, financial reporting, website analytics, or many other types or kinds of data. A client's use for the data may also affect the configuration of the data management system used to store the data. For instance, for certain types of data analysis and other operations, such as those that aggregate large sets of data from small numbers of columns within each row, a columnar database table may provide more efficient performance. In other words, column information from database tables may be stored into data blocks on disk, rather than storing entire rows of columns in each data block (as in traditional database schemes). The following discussion describes various embodiments of a relational columnar database system. However, various versions of the components discussed below may be equally adapted to implement embodiments for various other types of relational database systems, such as row-oriented database systems. Therefore, the following examples are not intended to be limiting as to various other types or formats of database systems.

In some embodiments, storing table data in such a columnar fashion may reduce the overall disk I/O requirements for various queries and may improve analytic query performance. For example, storing database table information in a columnar fashion may reduce the number of disk I/O requests performed when retrieving data into memory to perform database operations as part of processing a query (e.g., when retrieving all of the column field values for all of the rows in a table) and may reduce the amount of data that needs to be loaded from disk when processing a query. Conversely, for a given number of disk requests, more column field values for rows may be retrieved than is necessary when processing a query if each data block stored entire table rows. In some embodiments, the disk requirements may be further reduced using compression methods that are matched to the columnar storage data type. For example, since each block contains uniform data (i.e., column field values that are all of the same data type), disk storage and retrieval requirements may be further reduced by applying a compression method that is best suited to the particular column data type. In some embodiments, the savings in space for storing data blocks containing only field values of a single column on disk may translate into savings

in space when retrieving and then storing that data in system memory (e.g., when analyzing or otherwise processing the retrieved data).

Database service 210 may be implemented by a large collection of computing devices, such as customized or off-the-shelf computing systems, servers, or any other combination of computing systems or devices, such as the various types of systems 2000 described below with regard to FIG. 12. Different subsets of these computing devices may be controlled by control plane 220. Control plane 220, for example, may provide a cluster control interface to clients or users who wish to interact with the processing clusters, such as processing cluster(s) 232a, 232b, and 232c managed by control plane 220. For example, control plane 220 may generate one or more graphical user interfaces (GUIs) for clients, which may then be utilized to select various control functions offered by the control interface for the processing clusters 232 hosted in the database service 210. Control plane 220 may provide or implement access to various metrics collected for the performance of different features of database service 210, including processing cluster performance, in some embodiments.

Control plane 220 may also implement various systems to manage or implement database service 210 features. For example, control plane 220 may implement datashare metadata service 222. These systems may be used to implement managed datashares accessible across provider network regions 200. Data used to implement these features, such as datashare permission data 274 may be maintained in separate data storage service(s) 270, in some embodiments.

Database service 210 may also implement data exchange service 224 configured to manage authorization of data consumers. In some embodiments, data exchange service 224 may be implemented outside of database service 210 to provide additional authorization management for various other services offered by provider network 200. Database service 210 may delegate management of datashare permission data 274 to data exchange service 224.

As discussed above, various clients (or customers, organizations, entities, or users) may wish to store and manage data using a database service 210. Processing clusters 232 may respond to various requests, including write/update/store requests (e.g., to write data into storage) or queries for data (e.g., such as a Server Query Language request (SQL) for particular data). For example, multiple users or clients may access a processing cluster to obtain data warehouse services.

For databases manually managed by users, database service 210 may provide database endpoints directly to the clusters which allow the users to manage in order to implement client applications that send requests and other messages directly to a particular cluster. Database endpoints, for example, may be a network endpoint associated with a particular network address, such as a URL, which points to a resource, such as processing clusters 232 that are attached to the database for query processing. For instance, a client may be given the network endpoint "http://mycluster.com" to send various request messages to. Multiple clients (or users of a particular client) may be given a database endpoint for the same database. Various security features may be implemented to prevent unauthorized users from accessing the databases.

In at least some embodiments, database service 210 may implement proxy service 240 to provide access to databases (e.g., data warehouses) hosted in database service 210. For databases managed by database service 210, database service 210 may provide database endpoints 242 (e.g., network

endpoints) for a hosted database. Database endpoints **242** may not provide direct access to a particular processing cluster **232**, as the processing cluster used to respond to such requests (e.g., queries) may change according to the various scaling techniques. Instead, client applications may utilize the database endpoint **242** for a database to be included in various client applications or other communications for database access so that proxy service **240** can direct the requests to the appropriate processing cluster without the client application having to be altered every time a change in processing cluster (e.g., scaling operations) are performed by database service **210**. In this way, database service **210** can perform scaling and other management operations without interfering with client applications.

Processing clusters, such as processing clusters **232a**, **232b**, and **232c**, hosted by database service **210** may provide an enterprise-class database query and management system that allows users to send data processing requests to be executed by the clusters **232**, such as by sending a query. Processing clusters **232** may perform data processing operations with respect to data stored locally in a processing cluster, as well as remotely stored data. For example, data storage service **270** implemented by provider network **200** that stores remote data, such as backups or other data of a database stored in a cluster. In some embodiments, database data **272** may not be stored locally in a processing cluster **232** but instead may be stored in data storage service **270** (e.g., with data being partially or temporarily stored in processing cluster **232** to perform queries). Queries sent to a processing cluster **23** (or routed/redirect/assigned/allocated to processing cluster(s)) may be directed to local data stored in the processing cluster and/or remote data. Therefore, processing clusters may implement local data processing, such as local data processing, (discussed below with regard to FIG. 5) to plan and execute the performance of queries with respect to local data in the processing cluster, as well as a remote data processing client.

Database service **210** may implement different types or configurations of processing clusters. For example, different configurations A **230a**, B **230b**, and C **230c**, may utilize various different configurations of computing resources, including, but not limited to, different numbers of computational nodes, different processing capabilities (e.g., processor size, power, custom or task-specific hardware, such as hardware optimized to perform different operations, such as regular expression searching, or other data processing operations), different amounts of memory, different networking capabilities, and so on. Thus, for some queries, different configurations **230** of processing cluster **232** may offer different execution times. Different configurations **230** of processing clusters **232** may be maintained in different pools of available processing clusters to be attached to a database. Attached processing clusters may then be made exclusively assigned or allocated for the use of performing queries to the attached database, in some embodiments. The number of processing clusters **232** attached to a database may change over time according to the selection techniques discussed below.

In some embodiments, database service **210** may have at least one processing cluster attached to a database, which may be the “primary cluster.” Primary clusters may be reserved, allocated, permanent, or otherwise dedicated processing resources that store and/or provide access to a database for a client, in some embodiments. Primary clusters, however, may be changed. Techniques to resize or change to a different configuration of a primary cluster may be performed, in some embodiments. The available process-

ing clusters that may also be attached, as determined, to a database may be maintained (as noted earlier) in different configuration type pools, which may be a set of warmed, pre-configured, initialized, or otherwise prepared clusters which may be on standby to provide additional query performance capacity for a primary cluster. Control plane **220** may manage cluster pools by managing the size of cluster pools (e.g., by adding or removing processing clusters based on demand).

As databases are created, updated, and/or otherwise modified, snapshots, copies, or other replicas of the database at different states may be stored separate from database service **210** in data storage service **250**, in some embodiments. For example, a leader node, or other processing cluster component, may implement a backup agent or system that creates and store database backups for a database to be stored as database data **272** in data storage service **270**. Database data **272** may include user data (e.g., tables, rows, column values, etc.) and database metadata (e.g., information describing the tables which may be used to perform queries to a database, such as schema information, data distribution, range values or other content descriptors for filtering out portions of a table from a query, etc.). A timestamp or other sequence value indicating the version of database data **272** may be maintained in some embodiments, so that the latest database data **272** may, for instance, be obtained by a processing cluster in order to perform queries. In at least some embodiments, database data **272** may be treated as the authoritative version of data, and data stored in processing clusters **232** for local processing as a cached version of data.

Data storage service **270** may implement different types of data stores for storing, accessing, and managing data on behalf of clients **250** as a network-based service that enables clients **250** to operate a data storage system in a cloud or network computing environment. Data storage service(s) **270** may also include various kinds of object or file data stores for putting, updating, and getting data objects or files. For example, one data storage service **270** may be an object-based data store that allows for different data objects of different formats or types of data, such as structured data (e.g., database data stored in different database schemas), unstructured data (e.g., different types of documents or media content), or semi-structured data (e.g., different log files, human-readable data in different formats like JavaScript Object Notation (JSON) or Extensible Markup Language (XML)) to be stored and managed according to a key value or other unique identifier that identifies the object.

In at least some embodiments, data storage service(s) **270** may be treated as a data lake. For example, an organization may generate many different kinds of data, stored in one or multiple collections of data objects in a data storage service **270**. The data objects in the collection may include related or homogenous data objects, such as database partitions of sales data, as well as unrelated or heterogeneous data objects, such as audio files and web site log files. Data storage service(s) **270** may be accessed via programmatic interfaces (e.g., APIs) or graphical user interfaces.

Generally speaking, clients **250** may encompass any type of client that can submit network-based requests to provider network **200** via network **260**, including requests for storage services (e.g., a request to create a datashare at a database service **210**, or a request to create, read, write, obtain, or modify data in data storage service(s) **270**, etc.). For example, a given client **250** may include a suitable version of a web browser or may include a plug-in module or other type of code module that can execute as an extension to or within an execution environment provided by a web

13

browser. Alternatively, a client 250 may encompass an application such as a database application (or user interface thereof), a media application, an office application or any other application that may make use of database service(s) 210 or storage resources in data storage service(s) 270 to store and/or access the data to implement various applications. In some embodiments, such an application may include sufficient protocol support (e.g., for a suitable version of Hypertext Transfer Protocol (HTTP)) for generating and processing network-based services requests without necessarily implementing full browser support for all types of network-based data. That is, client 250 may be an application that can interact directly with provider network 200. In some embodiments, client 250 may generate network-based services requests according to a Representational State Transfer (REST)-style network-based services architecture, a document- or message-based network-based services architecture, or another suitable network-based services architecture.

In some embodiments, a client 250 may provide access to provider network 200 to other applications in a manner that is transparent to those applications. For example, client 250 may integrate with an operating system or file system to provide storage on one of data storage service(s) 270 (e.g., a block-based storage service). However, the operating system or file system may present a different storage interface to applications, such as a conventional file system hierarchy of files, directories and/or folders. In such an embodiment, applications may not need to be modified to make use of the storage system service model. Instead, the details of interfacing to the data storage service(s) 270 may be coordinated by client 250 and the operating system or file system on behalf of applications executing within the operating system environment. Similarly, a client 250 may be an analytics application that relies upon data processing service(s) 210 to execute various queries for data already ingested or stored in the data processing service (e.g., such as data maintained in a data warehouse service).

Clients 250 may convey network-based services requests (e.g., access requests to read or write data may be directed to data in data storage service(s) 270, or operations, tasks, or jobs, such as queries, being performed as part of data processing service(s) 210) to and receive responses from provider network 200 via network 260. In various embodiments, network 260 may encompass any suitable combination of networking hardware and protocols necessary to establish network-based communications between clients 250 and provider network 200. For example, network 260 may generally encompass the various telecommunications networks and service providers that collectively implement the Internet. Network 260 may also include private networks such as local area networks (LANs) or wide area networks (WANs) as well as public or private wireless networks. For example, both a given client 250 and provider network 200 may be respectively provisioned within enterprises having their own internal networks. In such an embodiment, network 260 may include the hardware (e.g., modems, routers, switches, load balancers, proxy servers, etc.) and software (e.g., protocol stacks, accounting software, firewall/security software, etc.) necessary to establish a networking link between given client 250 and the Internet as well as between the Internet and provider network 200. It is noted that in some embodiments, clients 250 may communicate with provider network 200 using a private network rather than the public Internet. In some embodiments, clients of data processing services 210 and/or data storage service(s) 270 may be implemented within provider network

14

200 (e.g., an application hosted on a virtual computing resource that utilizes a data processing service 210 to perform database queries) to implement various application features or functions and thus various features of client(s) 250 discussed above may be applicable to such internal clients as well.

FIG. 3 is a logical block diagram illustrating interactions between control plane components to share database data, according to some embodiments. Users may create datashares, grant permission for datashares, accept permissions for data shares, and delegate authorization management to a data exchange service. In order to support these interactions, different instantiations of database service 210 systems may communicate and work to support these features.

For example, as illustrated in FIG. 3, producer processing cluster 310 may produce data to be stored in a database, and consumer processing cluster 350 may access or consume the data stored in the database. These processing clusters may each implement leader nodes, such as leader node 312 and leader node 352, which may support interactions with client applications and other systems, such as control plane systems datashare metadata service 332, metadata proxy service 322, and datashare metadata service 334.

Datashare metadata service 332 in region 302 may manage interactions with datashare permission data 342. In this way, requests to create a datashare, authorize accounts, users, organizations, or other entities to access the datashare, conditions or limitations on the datashare, may be read, updated, or otherwise accessed in order to facilitate datashares. As indicated at 340, consumer processing cluster 350 may obtain datashare permission data 342 managed by data exchange service 380.

Leader nodes, such as leader nodes 312 and 352, may communicate through metadata proxy service 322. In this way, metadata may be accessed, as indicated at 320, without providing one cluster with direct access to another cluster, providing security safeguards. In some situations, leader node 312 may send a response to a request for access metadata and permission data based on authentication by one or more of metadata proxy service 322 or data exchange service 380, as indicated at 390.

Shared database data 362 may be stored in data object storage service 360 (e.g., similar to a data storage service 270 discussed above that utilizes an object-based store). Note that other data storage services like those discussed above with regard to data storage service 270 can be used in other embodiments. Data access 370 may be provided for shared database data 362 through requests to data object storage service 360 for consumer processing cluster 350. When data is used, data object storage service 362 can, for example, requests data blocks or other portions of shared database data 362 to return to consumer processing cluster 350 for use in performing queries.

In order to facilitate communications, different types of communication techniques may be implemented. FIG. 4 is a logical block diagram illustrating a network architecture for implementing communications between leader nodes to share database metadata, according to some embodiments. In FIG. 4, communications between leader node 412 of producer processing cluster 410 and leader node 452 of consumer processing cluster 450 may be facilitated through virtual private clouds and transit gateways.

A virtual private cloud (VPC) may be implemented as a logically isolated network within or portion of provider network 200, in order to allow a customer, system, service or user associated to specify the configuration within the

15

portion of the provider network, utilizing features such as specifying a range of network addresses (e.g., via a CIDR block), creation of overlay networks, such as subnets (which may be specified with different CIDR blocks), create and/or manage network route tables, and gateways as well as associate route tables for ingress traffic to a logically isolated network.

A transit gateway, such as transit gateways **450**, **452**, **454**, and **456**, may be a virtual traffic hub implemented as part of a network system or service of provider network **200** that may connect to multiple logically isolated networks, which may be programmatically attached in a hub-and-spoke configuration to a transit gateway, in various embodiments, so that the routing/forwarding of network packets from one attached logically isolated network to another is managed by nodes of a transit gateway based on metadata and/or policies provided by the creating/using clients, applications, entities, and/or systems.

Logically isolated networks attached to transit gateway (e.g., indicated by the arrows to transit gateways from VPCs, like VPCs **432**, **434**, and **442**) may allow network configuration settings (e.g., network addresses assigned to resources within a given network, subnet configurations, security settings and the like) to be set independently for the different networks, without having to take other networks' configuration settings into account. For example, a range of isolated network addresses selected for resources within one isolated network may happen to overlap with a range of private network addresses selected for resources within another isolated network in various embodiments, since the two address ranges are selected independently. According to some embodiments, the metadata taken into consideration at a transit gateway set up on behalf of a given client to manage traffic flowing between various isolated networks may include multiple route tables provided by the client (and/or route tables generated at the transit gateway based on other input provided by the client, such as forwarding information base (FIB) entries from which route table entries may be derived.) After the appropriate route tables have been associated and populated with entries, traffic may be allowed to start flowing between the isolated networks via transit gateways in various embodiments.

Utilizing VPCs and transit gateways, private network communications for access metadata and providing access credentials may be implemented while also prevent direct communication between leader nodes **412** and **452**. For example, producer processing cluster **410** may be implemented as part of the database service VPC **432**, which may utilize transit gateway **452** to route traffic to a VPC for the cross-region metadata proxy service **442** for region **402**. Metadata proxy service **442** may then be attached to different transit gateways attached to corresponding cross-region metadata proxy service VPCs in other regions, such as via transit gateway **456** to database storage **456** and transit gateway **450** to database service VPC **434**. Likewise, consumer processing cluster **450** may be implemented as part of the database service VPC **434**.

In order to implement managed datashares, the network architecture may utilize various routing table features of transit gateways. For example, connect any two VPCs in different AWS Regions, as long as they have distinct, non-overlapping CIDR blocks. This ensures that all of the private IP addresses are unique and allows all of the resources in the VPCs to address each other without the need for any form of network address translation. In this embodiment, a special cross-region metadata proxy service cell per region may be created. The cross-region metadata proxy

16

service VPC may still be peered with regional service VPC (e.g., for database service **210** within a region, like VPC **432** and **434**), and it will also be peered with cross-region metadata proxy service VPC in all the other regions to form a mesh across all other regions (e.g., as illustrated by the use of transit gateways to peer regions **402**, **404**, and **406**). Each cross-region metadata proxy service VPC may have a unique CIDR range that does not conflict with any other region, in some embodiments.

For determining CIDR allocation, some features of a routing solution may ensure that cross region metadata proxy service VPCs within a region have non overlapping CIDRs, the cross-region metadata proxy service VPC are non-overlapping across all regions, and limit CIDR reservations from a regional service VPC. For example, in some embodiments a reservation of 10.20.0.0/16 from service VPC to be distributed by cross-region metadata proxy service VPC within the region may be made to limit CIDR reservations. Continuing with the example, the lower 3 bits of the third IP decimal may be used to represent the cell ID, e.g., 000→cell **1**, 001→cell **2**. This leaves room for 8 cross-region metadata proxy service VPC cells in a region. A mapping may be maintained between a region code to the higher 5 bits of the third IP decimal (e.g., region **402**→00000, region **404**→00001, etc.). This leaves room for 25=32 regions. When further regions are added, a reservation of a new/16 CIDR block from a service VPC can be made, providing room for another 32 regions.

Note that other private communication techniques can be utilized, and thus the previous discussion is not intended to be limiting to other private communication techniques between processing clusters.

FIG. 5 is a logical block diagram illustrating an example processing cluster of a database service, according to some embodiments. Processing cluster **500** may be query processing cluster, like processing clusters **232** discussed above with regard to FIG. 2, that distributes execution of a query among multiple computational (which may also be referred to as "compute") nodes. As illustrated in this example, a processing cluster **500** may include a leader node **510** and compute nodes **520a**, **520b**, and **520n**, which may communicate with each other over an interconnect (not illustrated). Leader node **510** may implement query planning **512** to generate query plan(s), query execution **514** for executing queries on processing cluster **500** that perform data processing that can utilize remote query processing resources for remotely stored data (e.g., by utilizing one or more query execution slot(s)/queue(s) **517**). As described herein, each node in a primary processing cluster **500** may include attached storage, such as attached storage **522a**, **522b**, and **522n**, on which a database (or portions thereof) may be stored on behalf of clients (e.g., users, client applications, and/or storage service subscribers).

Note that in at least some embodiments, query processing capability may be separated from compute nodes, and thus in some embodiments, additional components may be implemented for processing queries. Additionally, it may be that in some embodiments, no one node in processing cluster **500** is a leader node as illustrated in FIG. 5, but rather different nodes of the nodes in processing cluster **500** may act as a leader node or otherwise direct processing of queries to data stored in processing cluster **500**. While nodes of processing cluster may be implemented on separate systems or devices, in at least some embodiments, some or all of processing cluster may be implemented as separate virtual nodes or instance on the same underlying hardware system (e.g., on a same server).

17

Leader node **510** may manage communications with clients, such as clients **250** discussed above with regard to FIG. 2. As discussed above with regard to FIG. 3, leader node **510** may communicate with proxy service **240** and may receive query **501** and return query results **503** to proxy service **240** (instead of communicating directly with a client application). Alternatively, in those embodiments where leader node **510** implements database query queue **320** and query routing **330** as a primary cluster, then leader node **510** may act as the proxy for other, secondary clusters, attached to the database, and may return query results directly to a client application.

Leader node **510** may be a node that receives a query **501** from various client programs (e.g., applications) and/or subscribers (users) (either directly or routed to leader node **510** from proxy service **240**), then parses them and develops an execution plan (e.g., query plan(s)) to carry out the associated database operation(s). More specifically, leader node **510** may develop the series of steps necessary to obtain results for the query. Query **501** may be directed to data that is stored both locally within processing cluster **500** (e.g., at one or more of compute nodes **520**) and data stored remotely. Leader node **510** may also manage the communications among compute nodes **520** instructed to carry out database operations for data stored in the processing cluster **500**. For example, node-specific query instructions **504** may be generated or compiled code by query execution **514** that is distributed by leader node **510** to various ones of the compute nodes **520** to carry out the steps needed to perform query **501**, including executing the code to generate intermediate results of query **501** at individual compute nodes may be sent back to the leader node **510**. Leader node **510** may receive data and query responses or results from compute nodes **520** in order to determine a final result **503** for query **501**.

A database schema, data format and/or other metadata information for the data stored among the compute nodes, such as the data tables stored in the cluster, may be managed and stored by leader node **510**. Query planning **512** may account for remotely stored data by generating node-specific query instructions that include remote operations to be directed by individual compute node(s). Although not illustrated, in some embodiments, a leader node may implement burst manager to send **506** a query plan generated by query planning **512** to be performed at another attached processing cluster and return results **508** received from the burst processing cluster to a client as part of results **503**. Query planning **512** may rely upon metadata updated via metadata requests **590** to metadata proxy services to perform queries on datashares.

In at least some embodiments, a result cache **519** may be implemented as part of leader node **510**. For example, as query results are generated, the results may also be stored in result cache **519** (or pointers to storage locations that store the results either in primary processing cluster **500** or in external storage locations), in some embodiments. Result cache **519** may be used instead of other processing cluster capacity, in some embodiments, by recognizing queries which would otherwise be sent to another attached processing cluster to be performed that have results stored in result cache **519**. Various caching strategies (e.g., LRU, FIFO, etc.) for result cache **519** may be implemented, in some embodiments. Although not illustrated in FIG. 5, result cache **519** could be stored in other storage systems (e.g., other storage services, such as a NoSQL database) and/or could store sub-query results.

18

Processing cluster **500** may also include compute nodes, such as compute nodes **520a**, **520b**, and **520n**. Compute nodes **520**, may for example, be implemented on servers or other computing devices, such as those described below with regard to computer system **2000** in FIG. 12, and each may include individual query processing “slices” defined, for example, for each core of a server’s multi-core processor, one or more query processing engine(s), such as query engine(s) **524a**, **524b**, and **524n**, to execute the instructions **504** or otherwise perform the portions of the query plan assigned to the compute node. Query engine(s) **524** may access a certain memory and disk space in order to process a portion of the workload for a query (or other database operation) that is sent to one or more of the compute nodes **520**. Query engine **524** may access attached storage, such as **522a**, **522b**, and **522n**, to perform local operation(s), such as local operations **518a**, **518b**, and **518n**. For example, query engine **524** may scan data in attached storage **522**, access indexes, perform joins, semi joins, aggregations, or any other processing operation assigned to the compute node **520**.

Query engine **524a** may also direct the execution of remote data processing operations, by providing remote operation(s), such as remote operations **516a**, **516b**, and **516n**, to remote data processing clients, such as remote data processing client **526a**, **526b**, and **526n**. Remote data processing clients **526** may be implemented by a client library, plugin, driver or other component that sends request sub-queries to be performed by data storage service **220** or requests to for data, **532a**, **532b**, and **532n**. As noted above, in some embodiments, remote data processing clients **526** may read, process, or otherwise obtain data **534a**, **534b**, and **534c**, in response from database data **540** in data storage service **270**, which may further process, combine, and or include them with results of location operations **518**. This database data **540** may, in some embodiments, be a datashare in provider network **200**.

Compute nodes **520** may send intermediate results from queries back to leader node **510** for final result generation (e.g., combining, aggregating, modifying, joining, etc.). Remote data processing clients **526** may retry data requests **532** that do not return within a retry threshold.

Attached storage **522** may be implemented as one or more of any type of storage devices and/or storage system suitable for storing data accessible to the compute nodes, including, but not limited to: redundant array of inexpensive disks (RAID) devices, disk drives (e.g., hard disk drives or solid state drives) or arrays of disk drives such as Just a Bunch Of Disks (JBOD), (used to refer to disks that are not implemented according to RAID), optical storage devices, tape drives, RAM disks, Storage Area Network (SAN), Network Access Storage (NAS), or combinations thereof. In various embodiments, disks may be formatted to store database tables (e.g., in column-oriented data formats or other data formats).

Although FIGS. 2-5 have been described and illustrated in the context of a provider network implementing a database service, like a data warehousing service, the various components illustrated and described in FIGS. 2-5 may be easily applied to other database engines that are hosted in different regions of a provider network. As such, FIGS. 2-5 are not intended to be limiting as to other embodiments of privately sharing database data.

FIG. 6 is a flowchart diagram illustrating a method **600** for establishing a datashare for a database, according to some embodiments. The method **600** may be implemented by producer database engine **120** of FIG. 1, producer process-

ing cluster **310** of FIG. **3**, producer processing cluster **410** of FIG. **4**, or processing cluster **500** of FIG. **5**, according to various embodiments.

The method **600** may include receiving, at a producer database engine of a database service, a creation request to create a datashare for a database managed by the producer database engine, at **602**. The producer database engine may be configured to enable sharing of the database with other entities, such as a consumer database engine that is separate from the producer database engine.

The method **600** may also include delegating, to a data exchange service, authorization management to access the datashare, at **604**. The data exchange service may be configured to authorize data consumers of the datashare without oversight from the producer database engine. For example, the data exchange service may be able to add or remove data consumers while the producer database engine is unable to change the data consumers that have been added by the data exchange service. The method **600** may include receiving, from a consumer database engine, a request to obtain metadata used to perform a query to the database of the datashare, at **706**. Receiving the request may occur at a subsequent time from delegation to the data exchange service.

The method **600** may further include determining that the data exchange service authorized the consumer database engine to access the datashare based on permission data for the datashare managed by the data exchange service, at **608**. In some embodiments, the data exchange service may maintain a list of authorized data consumers such that the producer database engine may query the data exchange service to verify the consumer database engine. In other embodiments, the data exchange service may inform the producer database engine of the list of authorized data consumers such that the producer database engine may have a record indicating that the consumer database engine is authorized by the data exchange service. The method **600** may conclude by returning the metadata to the consumer database engine, at **710**. The producer database engine may be configured to return the metadata in response to a determination that the data exchange service has authorized the consumer database engine.

FIG. **7** illustrates a flowchart diagram for a method **700** for delegating authorization management to a data exchange service, according to some embodiments. The method **700** may be implemented by producer database engine **120** of FIG. **1**, producer processing cluster **310** of FIG. **3**, producer processing cluster **410** of FIG. **4**, or processing cluster **500** of FIG. **5**, according to various embodiments.

The method **700** may include receiving a request, at a database engine, to create a datashare for a database managed by the database engine, at **702**. The request may indicate data to be included as part of the database. The request may indicate that the database is to be made available as a datashare such that consumer database engines may access the database via the datashare. The method **700** may also include updating metadata for the database to include the datashare at the database engine, at **704**. In some embodiments, the metadata may be managed by a control plane for the database. In other embodiments, the metadata may be managed by a metadata proxy service.

The method **700** may include receiving a request to delegate authorization management of the datashare to a data exchange service of a provider network that hosts the database, at **706**. The authorization management may be delegated to the data exchange service such that the data exchange service may manage adding or removing autho-

rized consumers for the datashare. The data exchange service may be separate from the database engine, in various embodiments. The method **700** may further include updating permission data for databases to include permission to the data exchange service to grant access to the database to consumer database engines, at **708**. The permission data may include records for authorized consumer database engines.

FIG. **8** illustrates a flowchart diagram for a method **800** for managing requests to access a shared database, according to some embodiments. The method **800** may be implemented by a metadata proxy service corresponding to data exchange service **150** of FIG. **1**, datashare metadata service **222** of FIG. **2**, metadata proxy service **322** of FIG. **3**, metadata proxy service VPC **442** of FIG. **4**, according to various embodiments.

The method **800** may include receiving, at a metadata proxy service, a request to access a database hosted in a database service of a provider network with a consumer account with a database engine of the provider network, the database being managed by a producer database engine, at **802**. The request may be sent by the database engine associated with the consumer account. The database engine may be separate from the producer database engine.

The method **800** may include accessing permission data managed by a data exchange service for databases hosted in the database service, at **804**. The permission data may be stored by the data exchange service in some embodiments. In other embodiments, the permission data may be stored by the producer database engine to indicate which accounts have been authorized by the data exchange service. The metadata proxy service may store a list of authorized accounts that have been authorized by the data exchange service.

The method **800** may include determining whether permission has been granted to the consumer account by the data exchange service, at **806**. Permission may be determined to have been granted based on the consumer account being indicated as authorized in the permission data. Based on a determination that permission has not been granted, the method **800** may include denying the request to access the database, at **808**. Based on a determination that permission has been granted, the method **800** may include sending, to the producer database engine, the request to access the database, at **810**. In some embodiments, the request may further include at least a portion of the permission data and metadata for the database. The producer database engine may be configured to obtain requested data from the database in accordance with the request, in various embodiments.

FIG. **9** illustrates a flowchart diagram depicting a method **900** for confirming application of changes to a datashare, according to some embodiments. The method **900** may be implemented by a database service, such as database service **110** of FIG. **1**, database service **210** of FIG. **2**, data object storage service **360** of FIG. **3**, database storage **456** of FIG. **4**, or data storage service **570** of FIG. **5**, according to various embodiments.

The method **900** may include receiving, at a database service, a change notification indicating an attempt of one or more changes by a producer database engine to a datashare of a database, at **902**. The one or more changes may be destructive operations that may modify accessibility of the datashare by data consumers. The method **900** may include requesting, from the producer database engine, a confirmation to apply the one or more changes to the datashare, at

904. The request for confirmation may include a one-time use code that must be returned to the database service as part of the confirmation.

The method 900 may include determining whether the database service has received the confirmation from the producer database engine, at 906. Any response from the producer database engine may be analyzed to determine whether the confirmation satisfies criteria for accepting the one or more changes. Based on a determination that the confirmation has not been received, the method 900 may further include rejecting the one or more changes to the datashare, at 907.

Based on a determination that the confirmation has been received, the method 900 may also include applying the one or more changes to the datashare, at 908. The datashare may be modified according to the one or more changes. For example, access to the datashare may be modified such that accessible data of the database may be reduced to consumers. The method 900 may also include updating metadata for the datashare based on the one or more changes to the datashare, at 910. The metadata may be used by a data exchange service to identify what data elements are shared according to the datashare.

The methods described herein may in various embodiments be implemented by any combination of hardware and software. For example, in one embodiment, the methods may be implemented by a computer system (e.g., a computer system as in FIG. 10) that includes one or more processors executing program instructions stored on a computer-readable storage medium coupled to the processors. The program instructions may implement the functionality described herein (e.g., the functionality of various servers and other components that implement the network-based virtual computing resource provider described herein). The various methods as illustrated in the figures and described herein represent example embodiments of methods. The order of any method may be changed, and various elements may be added, reordered, combined, omitted, modified, etc.

Embodiments of privately sharing database data across provider network regions as described herein may be executed on one or more computer systems, which may interact with various other devices. One such computer system is illustrated by FIG. 10. In different embodiments, computer system 2000 may be any of various types of devices, including, but not limited to, a personal computer system, desktop computer, laptop, notebook, or netbook computer, mainframe computer system, handheld computer, workstation, network computer, a camera, a set top box, a mobile device, a consumer device, video game console, handheld video game device, application server, storage device, a peripheral device such as a switch, modem, router, or in general any type of computing node, compute node, computing device, compute device, or electronic device.

In the illustrated embodiment, computer system 2000 includes one or more processors 2010 coupled to a system memory 2020 via an input/output (I/O) interface 2030. Computer system 2000 further includes a network interface 2040 coupled to I/O interface 2030, and one or more input/output devices 2050, such as cursor control device 2060, keyboard 2070, and display(s) 2080. Display(s) 2080 may include standard computer monitor(s) and/or other display systems, technologies or devices. In at least some implementations, the input/output devices 2050 may also include a touch- or multi-touch enabled device such as a pad or tablet via which a user enters input via a stylus-type device and/or one or more digits. In some embodiments, it is contemplated that embodiments may be implemented

using a single instance of computer system 2000, while in other embodiments multiple such systems, or multiple nodes making up computer system 2000, may host different portions or instances of embodiments. For example, in one embodiment some elements may be implemented via one or more nodes of computer system 2000 that are distinct from those nodes implementing other elements.

In various embodiments, computer system 2000 may be a uniprocessor system including one processor 2010, or a multiprocessor system including several processors 2010 (e.g., two, four, eight, or another suitable number). Processors 2010 may be any suitable processor capable of executing instructions. For example, in various embodiments, processors 2010 may be general-purpose or embedded processors implementing any of a variety of instruction set architectures (ISAs), such as the x86, PowerPC, SPARC, or MIPS ISAs, or any other suitable ISA. In multiprocessor systems, each of processors 2010 may commonly, but not necessarily, implement the same ISA.

In some embodiments, at least one processor 2010 may be a graphics processing unit. A graphics processing unit or GPU may be considered a dedicated graphics-rendering device for a personal computer, workstation, game console or other computing or electronic device. Modern GPUs may be very efficient at manipulating and displaying computer graphics, and their highly parallel structure may make them more effective than typical CPUs for a range of complex graphical algorithms. For example, a graphics processor may implement a number of graphics primitive operations in a way that makes executing them much faster than drawing directly to the screen with a host central processing unit (CPU). In various embodiments, graphics rendering may, at least in part, be implemented by program instructions that execute on one of, or parallel execution on two or more of, such GPUs. The GPU(s) may implement one or more application programmer interfaces (APIs) that permit programmers to invoke the functionality of the GPU(s). Suitable GPUs may be commercially available from vendors such as NVIDIA Corporation, Advanced Micro Devices (AMD), and others.

System memory 2020 may store program instructions and/or data accessible by processor 2010. In various embodiments, system memory 2020 may be implemented using any suitable memory technology, such as static random-access memory (SRAM), synchronous dynamic RAM (SDRAM), nonvolatile/Flash-type memory, or any other type of memory. In the illustrated embodiment, program instructions and data implementing desired functions, such as those described above are shown stored within system memory 2020 as program instructions 2025 and data storage 2035, respectively. In other embodiments, program instructions and/or data may be received, sent or stored upon different types of computer-accessible media or on similar media separate from system memory 2020 or computer system 2000. Generally speaking, a non-transitory, computer-readable storage medium may include storage media or memory media such as magnetic or optical media, e.g., disk or CD/DVD-ROM coupled to computer system 2000 via I/O interface 2030. Program instructions and data stored via a computer-readable medium may be transmitted by transmission media or signals such as electrical, electromagnetic, or digital signals, which may be conveyed via a communication medium such as a network and/or a wireless link, such as may be implemented via network interface 2040.

In one embodiment, I/O interface 2030 may coordinate I/O traffic between processor 2010, system memory 2020,

and any peripheral devices in the device, including network interface **2040** or other peripheral interfaces, such as input/output devices **2050**. In some embodiments, I/O interface **2030** may perform any necessary protocol, timing or other data transformations to convert data signals from one component (e.g., system memory **2020**) into a format suitable for use by another component (e.g., processor **2010**). In some embodiments, I/O interface **2030** may include support for devices attached through various types of peripheral buses, such as a variant of the Peripheral Component Interconnect (PCI) bus standard or the Universal Serial Bus (USB) standard, for example. In some embodiments, the function of I/O interface **2030** may be split into two or more separate components, such as a north bridge and a south bridge, for example. In addition, in some embodiments some or all of the functionality of I/O interface **2030**, such as an interface to system memory **2020**, may be incorporated directly into processor **2010**.

Network interface **2040** may allow data to be exchanged between computer system **2000** and other devices attached to a network, such as other computer systems, or between nodes of computer system **2000**. In various embodiments, network interface **2040** may support communication via wired or wireless general data networks, such as any suitable type of Ethernet network, for example; via telecommunications/telephony networks such as analog voice networks or digital fiber communications networks; via storage area networks such as Fibre Channel SANs, or via any other suitable type of network and/or protocol.

Input/output devices **2050** may, in some embodiments, include one or more display terminals, keyboards, keypads, touchpads, scanning devices, voice or optical recognition devices, or any other devices suitable for entering or retrieving data by one or more computer system **2000**. Multiple input/output devices **2050** may be present in computer system **2000** or may be distributed on various nodes of computer system **2000**. In some embodiments, similar input/output devices may be separate from computer system **2000** and may interact with one or more nodes of computer system **2000** through a wired or wireless connection, such as over network interface **2040**.

As shown in FIG. 10, memory **2020** may include program instructions **2025**, that implement the various methods and techniques as described herein, and data storage **2035**, comprising various data accessible by program instructions **2025**. In one embodiment, program instructions **2025** may include software elements of embodiments as described herein and as illustrated in the Figures. Data storage **2035** may include data that may be used in embodiments. In other embodiments, other or different software elements and data may be included.

Those skilled in the art will appreciate that computer system **2000** is merely illustrative and is not intended to limit the scope of the techniques as described herein. In particular, the computer system and devices may include any combination of hardware or software that can perform the indicated functions, including a computer, personal computer system, desktop computer, laptop, notebook, or netbook computer, mainframe computer system, handheld computer, workstation, network computer, a camera, a set top box, a mobile device, network device, internet appliance, PDA, wireless phones, pagers, a consumer device, video game console, handheld video game device, application server, storage device, a peripheral device such as a switch, modem, router, or in general any type of computing or electronic device. Computer system **2000** may also be connected to other devices that are not illustrated, or instead may operate

as a stand-alone system. In addition, the functionality provided by the illustrated components may in some embodiments be combined in fewer components or distributed in additional components. Similarly, in some embodiments, the functionality of some of the illustrated components may not be provided and/or other additional functionality may be available.

Those skilled in the art will also appreciate that, while various items are illustrated as being stored in memory or on storage while being used, these items or portions of them may be transferred between memory and other storage devices for purposes of memory management and data integrity. Alternatively, in other embodiments some or all of the software components may execute in memory on another device and communicate with the illustrated computer system via inter-computer communication. Some or all of the system components or data structures may also be stored (e.g., as instructions or structured data) on a computer-accessible medium or a portable article to be read by an appropriate drive, various examples of which are described above. In some embodiments, instructions stored on a non-transitory, computer-accessible medium separate from computer system **2000** may be transmitted to computer system **2000** via transmission media or signals such as electrical, electromagnetic, or digital signals, conveyed via a communication medium such as a network and/or a wireless link. Various embodiments may further include receiving, sending or storing instructions and/or data implemented in accordance with the foregoing description upon a computer-accessible medium. Accordingly, the present invention may be practiced with other computer system configurations.

It is noted that any of the distributed system embodiments described herein, or any of their components, may be implemented as one or more web services. In some embodiments, a network-based service may be implemented by a software and/or hardware system designed to support interoperable machine-to-machine interaction over a network. A network-based service may have an interface described in a machine-processable format, such as the Web Services Description Language (WSDL). Other systems may interact with the web service in a manner prescribed by the description of the network-based service's interface. For example, the network-based service may define various operations that other systems may invoke, and may define a particular application programming interface (API) to which other systems may be expected to conform when requesting the various operations.

In various embodiments, a network-based service may be requested or invoked through the use of a message that includes parameters and/or data associated with the network-based services request. Such a message may be formatted according to a particular markup language such as Extensible Markup Language (XML), and/or may be encapsulated using a protocol such as Simple Object Access Protocol (SOAP). To perform a web services request, a network-based services client may assemble a message including the request and convey the message to an addressable endpoint (e.g., a Uniform Resource Locator (URL)) corresponding to the web service, using an Internet-based application layer transfer protocol such as Hypertext Transfer Protocol (HTTP).

In some embodiments, web services may be implemented using Representational State Transfer ("RESTful") techniques rather than message-based techniques. For example, a web service implemented according to a RESTful technique may be invoked through parameters included within

25

an HTTP method such as PUT, GET, or DELETE, rather than encapsulated within a SOAP message.

The various methods as illustrated in the FIGS. and described herein represent example embodiments of methods. The methods may be implemented in software, hardware, or a combination thereof. The order of method may be changed, and various elements may be added, reordered, combined, omitted, modified, etc.

Various modifications and changes may be made as would be obvious to a person skilled in the art having the benefit of this disclosure. It is intended that the invention embrace all such modifications and changes and, accordingly, the above description to be regarded in an illustrative rather than a restrictive sense.

What is claimed is:

1. A system, comprising:

a plurality of computing devices, respectively comprising at least one processor and a memory, that implement a database service of a cloud service provider that hosts databases created via an interface of the database service;

wherein the database service comprises a control plane and a producer database engine;

wherein the producer database engine is configured to:

receive a creation request to create a datashare for a database managed by the producer database engine, wherein the creation request delegates, to a data exchange service separate from the database service, authorization management to access the datashare;

wherein the control plane is configured to:

update permission data for the datashare to indicate that authorization to the datashare is managed by the data exchange service, wherein the permission data is accessible to the data exchange service to authorize consumer database engines to access the datashare;

wherein data of the database is stored in a storage service, separate from the database service, that is accessible to both the producer database engine and the consumer database engines to perform queries to the database;

wherein the producer database engine is further configured to:

receive a request, from a consumer database engine, to obtain metadata comprising a schema of the database that includes information used to generate a plan to perform a query to the database of the datashare;

determine that the data exchange service authorized the consumer database engine to access the datashare based on the permission data for the datashare; and responsive to the determination that the data exchange service authorized the consumer database engine to access the datashare, return the metadata to the consumer database engine for the consumer database engine to generate the plan to perform the query to the database.

2. The system of claim 1, wherein the control plane is further configured to:

receive, from the data exchange service, consumer change information indicating a change in the consumer database engines authorized to access the data share; and update the permission data for the datashare according to the consumer change information.

3. The system of claim 1, wherein the control plane is further configured to:

send, to the data exchange service, further metadata for the datashare indicating that the datashare is accessible by the consumer database engines in accordance with the data exchange service.

26

4. The system of claim 1, wherein the producer database engine is configured to:

attempt to cause one or more changes to the datashare; and

wherein the database service is further configured to:

receive a change notification indicating the attempt of the one or more changes to the datashare; and request, from the producer database engine, a confirmation to apply the one or more changes to the datashare.

5. The system of claim 4, wherein the database service is configured to:

receive the confirmation from the producer database engine;

responsive to the confirmation,

apply the one or more changes to the datashare; and update further metadata for the datashare based on the one or more changes to the datashare.

6. A method, comprising:

creating, by a producer database engine of a database service that hosts databases created via an interface of the database service, a datashare for a database managed by the producer database, wherein the creation delegates authorization management to access the datashare to a datashare authentication manager separate from the database service;

receiving a request, from a consumer database engine at the producer database engine, to obtain metadata comprising a schema of the database that includes information used to generate a plan to perform a query to the database of the datashare;

wherein data of the database is stored in a storage service, separate from the database service, that is accessible to both the producer database engine and the consumer database engines to perform queries to the database; and

responsive to determining that the datashare authentication manager authorized the consumer database engine to access the datashare according to permission data for the datashare that is accessible to the data exchange service, returning the metadata to the consumer database engine for the consumer database engine to generate the plan to perform the query to the database in response to the request.

7. The method of claim 6, further comprising:

receiving, from the datashare authentication manager, consumer change information indicating a change in the consumer database engines authorized to access the datashare; and

updating the permission data for the datashare according to the consumer change information.

8. The method of claim 7, wherein the change in the consumer database engines comprises:

adding or removing a consumer database engine from the consumer database engines authorized to access the datashare.

9. The method of claim 6, further comprising:

sending, to the datashare authentication manager, further metadata for the datashare indicating that the datashare is accessible by the consumer database engines in accordance with the datashare authentication manager.

10. The method of claim 6, further comprising:

receiving a change notification indicating an attempt of one or more changes to the datashare; and requesting a confirmation to apply the one or more changes to the datashare.

27

11. The method of claim 10, further comprising:
 receiving the confirmation from the producer database engine;
 responsive to the confirmation,
 applying the one or more changes to the datashare; and
 updating further metadata for the datashare based on
 the one or more changes to the datashare.

12. The method of claim 10, wherein the one or more changes to the datashare comprise a destructive operation to the datashare, and wherein the confirmation is requested in response to determining that the one or more changes comprise the destructive operation.

13. The method of claim 6, further comprising:
 logging access to the datashare by the consumer database engine as being associated with the datashare authentication manager.

14. One or more computer-readable storage media storing instructions that, when executed on or across one or more processors, cause the one or more processors to:

create, by a producer database engine of a database service that hosts databases created via an interface of the database service, a datashare for a database managed by the producer database, wherein the creation delegates authorization management to access the datashare to a data exchange service separate from the database service;

responsive to a request, from a consumer database engine at the producer database engine, to obtain metadata comprising a schema of the database that includes information used to generate a plan to perform a query to the database of the datashare, determine whether the data exchange service authorized the consumer database engine to access the datashare according to permission data for the datashare that is accessible to the data exchange service;

wherein data of the database is stored in a storage service, separate from the database service, that is accessible to both the producer database engine and the consumer database engines to perform queries to the database; and

responsive to a determination that the data exchange service authorized the consumer database engine, return the metadata to the consumer database engine for the consumer database engine to generate the plan to perform the query to the database in response to the request.

28

15. The one or more computer-readable storage media of claim 14, further comprising instructions that, when executed the on or across one or more processors, cause the one or more processors to:

in response to receiving, from the data exchange service, consumer change information indicating a change in the consumer database engines authorized to access the datashare, update the permission data for the datashare according to the consumer change information.

16. The one or more computer-readable storage media of claim 15, wherein the change in the consumer database engines comprises:

adding or removing a consumer database engine from the consumer database engines authorized to access the datashare.

17. The one or more computer-readable storage media of claim 14, further comprising instructions that, when executed the on or across one or more processors, cause the one or more processors to:

send, to the data exchange service, further metadata for the datashare indicating that the datashare is accessible by the consumer database engines in accordance with the data exchange service.

18. The one or more computer-readable storage media of claim 14, further comprising instructions that, when executed the on or across one or more processors, cause the one or more processors to:

detect an attempt of one or more changes to the datashare; and

requesting a confirmation to apply the one or more changes to the datashare.

19. The one or more computer-readable storage media of claim 18, further comprising instructions that, when executed the on or across one or more processors, cause the one or more processors to:

responsive to the confirmation from the producer database engine,

apply the one or more changes to the datashare; and
 update further metadata for the datashare based on the one or more changes to the datashare.

20. The one or more computer-readable storage media of claim 14, further comprising instructions that, when executed the on or across one or more processors, cause the one or more processors to:

log access to the datashare by the consumer database engine as being associated with the data exchange service.

* * * * *